École Nationale Supérieure d'Arts et Métiers — Meknès
Université Moulay Ismaïl
Filière : Intelligence Artificielle & Technologies des Données

RAPPORT DE STAGE
Projet de Fin d'Année (PFA)

Conception et réalisation d'un assistant RAG multimodal (texte + images) pour l'interrogation de documentation technique

Réalisé par Amine Amllal

Encadré par DARKAOUI Hatim

Entreprise d'accueil ALTEN

Durée du stage 8 semaines (mai – juillet 2026)

Année universitaire 2025–2026

Remerciements

Au terme de ce stage de Projet de Fin d'Année, je tiens à exprimer ma profonde gratitude à l'ensemble des personnes qui ont contribué, de près ou de loin, à son bon déroulement.

Mes remerciements s'adressent en premier lieu à M. DARKAOUI Hatim, mon encadrant au sein d'ALTEN, pour la confiance qu'il m'a accordée, la pertinence de ses orientations techniques et sa disponibilité tout au long de ces huit semaines. Ses retours réguliers m'ont permis de garder un cap clair entre l'ambition fonctionnelle du projet et les contraintes réelles d'un environnement industriel.

Je remercie également l'ensemble des équipes d'ALTEN pour leur accueil chaleureux et pour l'environnement de travail stimulant dans lequel j'ai pu évoluer. Travailler au contact d'ingénieurs confrontés quotidiennement à des problématiques concrètes de gestion de documentation technique a donné tout son sens à ce projet.

Ma reconnaissance va aussi au corps professoral de l'École Nationale Supérieure d'Arts et Métiers de Meknès, et en particulier aux enseignants de la filière Intelligence Artificielle & Technologies des Données, dont les enseignements ont constitué le socle théorique sur lequel ce travail s'est appuyé.

Enfin, je remercie ma famille et mes proches pour leur soutien constant.

Résumé

Ce rapport présente la conception et la réalisation d'un **assistant de recherche documentaire multimodal** fondé sur le paradigme RAG (*Retrieval-Augmented Generation*), développé au cours d'un stage PFA de huit semaines au sein d'ALTEN. L'objectif est de permettre à un ingénieur d'interroger en langage naturel un corpus de documents techniques (articles scientifiques, manuels d'ingénierie au format PDF ou Word) et d'obtenir des réponses *sourcées*, combinant extraits textuels cités et figures pertinentes affichées directement dans l'interface.

La chaîne de traitement réalisée couvre l'intégralité du cycle de vie documentaire : analyse structurelle des documents avec **Docling** (détection de mise en page, extraction des figures, reconstruction des tableaux), découpage sémantique par *HybridChunker*, description automatique des figures par un modèle de vision-langage (**Gemma-3 27B**), vectorisation unifiée des chunks textuels et des descriptions d'images par **mistral-embed**, puis indexation dans **ChromaDB**. La phase de restitution repose sur une **recherche hybride** combinant similarité dense et BM25 par fusion de rangs réciproques (RRF), un routage automatique de la question vers les groupes documentaires pertinents par LLM, et la génération de la réponse par **mistral-small-latest** avec citation systématique des sources. Les capacités sont exposées par une API REST **FastAPI** et une interface **React/TypeScript** sécurisée par **Clerk**.

La qualité du système a été évaluée avec le cadre **RAGAS**, en utilisant un juge LLM indépendant (Llama 3.3 70B). Les résultats obtenus (*context recall* de 1,00, *answer relevancy* de 0,93 et *faithfulness* moyenne de 0,83) valident l'efficacité du retrieval hybride et de l'indexation multimodale, tout en identifiant des axes d'amélioration précis.

Mots-clés : RAG multimodal, LLM, embeddings, recherche hybride, BM25, RRF, Docling, ChromaDB, Mistral AI, VLM, FastAPI, React, RAGAS.

Abstract

This report presents the design and implementation of a **multimodal Retrieval-Augmented Generation (RAG) assistant**, developed during an eight-week internship at ALTEN. The system enables engineers to query a corpus of technical documents (research papers and engineering manuals in PDF or Word format) in natural language, and to receive *grounded* answers combining cited text excerpts and the relevant figures displayed directly in the user interface.

The implemented pipeline covers the full document lifecycle : structural parsing with **Docling** (layout detection, figure extraction, table reconstruction), semantic chunking with the *HybridChunker*, automatic figure description using a vision-language model (**Gemma-3 27B**), unified vectorization of text chunks and image descriptions with **mistral-embed**, and indexing into **ChromaDB**. Retrieval combines dense similarity search and BM25 keyword matching through reciprocal rank fusion (RRF), augmented by automatic LLM-based routing of each question to the relevant document groups. Answers are generated by **mistral-small-latest** with systematic source citation. All capabilities are exposed through a **FastAPI** REST API and a **React/TypeScript** interface secured with **Clerk** authentication.

System quality was evaluated with the **RAGAS** framework, using an independent LLM judge (Llama 3.3 70B). The results (context recall of 1.00, answer relevancy of 0.93 and average faithfulness of 0.83) validate the effectiveness of hybrid retrieval and multimodal indexing, while identifying precise improvement paths.

Keywords : multimodal RAG, LLM, embeddings, hybrid search, BM25, RRF, Docling, ChromaDB, Mistral AI, VLM, FastAPI, React, RAGAS.

Table des matières

Remerciements i

Résumé ii

Abstract iii

Liste des figures v

Liste des tableaux vi

Liste des abréviations vii

Introduction générale 1

1 État de l'art des systèmes RAG multimodaux et positionnement du problème 3

1.1 Introduction . . . 3

1.2 Contexte : la documentation technique chez ALTEN . . . 3

1.2.1 L'entreprise d'accueil . . . 3

1.2.2 Problématique métier . . . 3

1.3 Concepts et notions théoriques clés . . . 4

1.3.1 Grands modèles de langue (LLM) . . . 4

1.3.2 Embeddings et recherche sémantique . . . 4

1.3.3 Recherche lexicale : BM25 . . . 5

1.3.4 Retrieval-Augmented Generation (RAG) . . . 5

1.3.5 Le découpage sémantique (chunking) . . . 5

1.3.6 Modèles de vision-langage (VLM) . . . 6

1.4 Approches existantes pour le RAG multimodal . . . 6

1.4.1 OCR seul . . . 6

1.4.2 Captioning par VLM (description textuelle) . . . 6

1.4.3 Embeddings multimodaux natifs . . . 6

1.5 Outils et solutions existants . . . 7
1.5.1 Cadres logiciels open source . . . 7
1.5.2 Analyse documentaire . . . 7
1.5.3 Bases vectorielles . . . 8
1.5.4 Solutions intégrées du marché . . . 8
1.6 Limites des solutions existantes . . . 8
1.7 Apport de la solution proposée . . . 8
1.8 Positionnement du projet . . . 9
1.9 Conclusion . . . 9

2 Analyse, conception et architecture de la solution 11
2.1 Introduction . . . 11
2.2 Présentation générale du projet . . . 11
2.3 Besoins fonctionnels . . . 12
2.4 Besoins non fonctionnels . . . 12
2.5 Principe général du pipeline . . . 13
2.5.1 Chaîne d'ingestion . . . 14
2.5.2 Chaîne de restitution . . . 14
2.6 Architecture globale du système . . . 14
2.7 Architecture du backend . . . 15
2.8 Architecture du frontend . . . 16
2.9 Communication et flux entre composants . . . 18
2.10 Modélisation UML . . . 18
2.11 Modèle de données . . . 18
2.12 Justification des choix technologiques . . . 21
2.13 Conclusion . . . 22

3 Réalisation, implémentation et validation 23
3.1 Introduction . . . 23
3.2 Environnement de développement . . . 23
3.2.1 Poste de travail et contraintes . . . 23
3.2.2 Outillage . . . 24
3.3 Implémentation des composants principaux . . . 24
3.3.1 Analyse documentaire et découpage (`docling_parser`) . . . 24
3.3.2 Description des figures par VLM (`hf_client`) . . . 24

3.3.3 Embeddings multi-backends . . . 25
3.3.4 Recherche hybride (`vector_store`) . . . 25
3.3.5 Routage automatique du périmètre (`auto-scope`) . . . 25
3.3.6 Génération contrainte et restitution multimodale . . . 26
3.3.7 Interface utilisateur . . . 26
3.4 Intégration des services tiers . . . 26
3.5 Détails techniques spécifiques à l'environnement contraint . . . 27
3.5.1 Traversée du proxy Zscaler : `truststore` . . . 27
3.5.2 Modèles Docling hors ligne . . . 27
3.5.3 Cache de conversations fail-open . . . 27
3.6 Sécurité applicative . . . 27
3.7 Tests et validation : évaluation RAGAS . . . 28
3.7.1 Méthodologie . . . 28
3.7.2 Résultats obtenus . . . 28
3.7.3 Analyse des résultats . . . 28
3.8 Scénario de démonstration . . . 29
3.9 Résultats obtenus . . . 30
3.10 Difficultés rencontrées . . . 30
3.11 Limites de la solution . . . 31
3.12 Conclusion . . . 31

Conclusion générale **32**

Perspectives **33**

Bibliographie et webographie **34**

A Principales routes de l'API REST **36**

Table des figures

1.1 Principe général du paradigme RAG : indexation hors ligne du corpus, puis récupération et génération contrainte à chaque question. 5

1.2 Les trois stratégies d'indexation des images d'un corpus : OCR seul, description par VLM, embeddings multimodaux natifs. 7

2.1 Pipeline fonctionnel complet : chaîne d'ingestion (à gauche) et chaîne de restitution (à droite), articulées autour de l'index unifié ChromaDB + BM25. 13

2.2 Architecture globale : SPA React, API FastAPI, couche de persistance embarquée et services d'IA externes. 15

2.3 Diagramme de paquetages du backend : dépendances entre routes, services et stockage. 16

2.4 Interface conversationnelle : réponse sourcée avec citations et figure affichée sous la réponse. 17

2.5 Vue de gestion du corpus : groupes documentaires, état d'ingestion en temps réel et actions par document. 17

2.6 Diagramme de séquence du flux d'ingestion d'un document. 18

2.7 Diagramme de séquence du flux de question–réponse sourcée. 19

2.8 Diagramme de cas d'utilisation de l'assistant RAG multimodal. 19

2.9 Modèle de données : tables SQLite, collection ChromaDB et stockage fichiers, reliés par l'identifiant de document. 20

3.1 Synthèse visuelle des quatre métriques RAGAS moyennes du système. 29

Liste des tableaux

1.1 Positionnement de la solution développée par rapport à l'existant. 9

2.1 Synthèse des besoins fonctionnels. 12
2.2 Synthèse des besoins non fonctionnels. 12
2.3 Choix technologiques et justifications. 21

3.1 Services externes intégrés et leur rôle. 26
3.2 Résultats de l'évaluation RAGAS (juge : Llama 3.3 70B via Groq). 28

A.1 Routes exposées par l'API FastAPI. 36

Liste des abréviations

API	*Application Programming Interface* : interface de programmation applicative
BM25	*Best Matching 25* : fonction de classement lexical par mots-clés
CORS	*Cross-Origin Resource Sharing* : partage de ressources entre origines
CPU	*Central Processing Unit* : processeur central
CRUD	*Create, Read, Update, Delete* : opérations de persistance élémentaires
JWKS	*JSON Web Key Set* : ensemble de clés publiques de vérification de jetons
JWT	*JSON Web Token* : jeton d'authentification signé
LLM	*Large Language Model* : grand modèle de langue
MLM	*Masked Language Modeling* : modélisation de langue masquée (pré-entraînement BERT)
OCI	*Oracle Cloud Infrastructure* : infrastructure cloud d'Oracle
OCR	*Optical Character Recognition* : reconnaissance optique de caractères
PFA	Projet de Fin d'Année
RAG	*Retrieval-Augmented Generation* : génération augmentée par récupération
RAGAS	*Retrieval-Augmented Generation Assessment* : cadre d'évaluation de systèmes RAG
REST	*Representational State Transfer* : style d'architecture d'API web
RRF	*Reciprocal Rank Fusion* : fusion de rangs réciproques
SPA	*Single-Page Application* : application web monopage
SQL	*Structured Query Language* : langage de requête structuré
TLS	*Transport Layer Security* : protocole de sécurisation des échanges
UI	*User Interface* : interface utilisateur
UML	*Unified Modeling Language* : langage de modélisation unifié
VLM	*Vision-Language Model* : modèle de vision-langage

Introduction générale

Les organisations d'ingénierie produisent et consomment des volumes considérables de documentation technique : articles de recherche, manuels d'équipements, guides de normes, schémas et spécifications. Cette connaissance, essentielle au travail quotidien des ingénieurs, reste pourtant difficile d'accès : elle est enfermée dans des fichiers PDF ou Word hétérogènes, dont une partie significative de l'information est portée par des *éléments visuels* (diagrammes d'architecture, schémas électriques, courbes de résultats, tableaux) que les moteurs de recherche classiques, fondés sur le texte seul, ignorent purement et simplement.

L'émergence des grands modèles de langue (LLM) a ouvert une voie nouvelle : le paradigme RAG (*Retrieval-Augmented Generation*) permet de coupler la capacité de synthèse d'un LLM à une base documentaire propre à l'organisation, en récupérant à chaque question les passages pertinents du corpus et en contraignant le modèle à y ancrer sa réponse. Le RAG répond ainsi aux deux limites majeures des LLM utilisés seuls : l'absence de connaissance des documents internes et la propension à l'hallucination. Toutefois, la majorité des systèmes RAG restent *unimodaux* : ils n'indexent que le texte, et une question dont la réponse se trouve dans une figure reste sans réponse.

C'est précisément le problème auquel s'attaque ce projet, réalisé dans le cadre d'un stage de Projet de Fin d'Année de huit semaines au sein d'**ALTEN**, leader mondial de l'ingénierie et du conseil en technologies. L'objectif du stage est de concevoir et de réaliser un **assistant RAG multimodal** capable d'interroger en langage naturel un corpus de documents techniques et de restituer des réponses sourcées mêlant extraits de texte et images pertinentes. Le projet couvre l'intégralité de la chaîne : ingestion et analyse structurelle des documents (Docling), description automatique des figures par un modèle de vision-langage, indexation vectorielle unifiée (ChromaDB), recherche hybride dense + lexicale, génération de réponses citées (Mistral AI), exposition par une API REST (FastAPI), interface de démonstration (React/TypeScript) et évaluation quantitative de la qualité des réponses (RAGAS).

Une particularité de ce stage mérite d'être soulignée dès l'introduction : le développement s'est déroulé sur un *poste professionnel verrouillé*, derrière un proxy d'inspection TLS (Zscaler), sans droits d'administration ni possibilité d'installer des services système. Ces contraintes, représentatives des environnements industriels réels, ont fortement structuré les choix techniques : préférence pour des API cloud accessibles, solutions embarquées sans serveur (SQLite, ChromaDB persistant, FakeRedis), modèles pré-téléchargés hors ligne ; ils constituent en eux-mêmes un retour d'expérience précieux.

Le présent rapport s'articule en trois chapitres. Le **chapitre 1** dresse l'état de l'art des systèmes RAG et de leurs extensions multimodales, analyse les approches et outils existants, et positionne la solution proposée. Le **chapitre 2** présente l'analyse des besoins, la conception et l'architecture du système : pipeline fonctionnel, architecture logicielle, modélisation UML, modèle de données et justification des choix technologiques. Le **chapitre 3** détaille la réalisation : environnement de développement contraint, implémentation des composants, intégration des services tiers, sécurité applicative, évaluation RAGAS, résultats obtenus, difficultés rencontrées et limites. Une conclusion générale et des perspectives d'évolution clôturent le document.

1 Chapitre

État de l'art des systèmes RAG multimodaux et positionnement du problème

1.1 Introduction

Ce premier chapitre pose le cadre scientifique et industriel du projet. Il présente d'abord le contexte de l'entreprise d'accueil et le domaine d'application (la recherche d'information dans la documentation technique d'ingénierie), puis introduit les concepts théoriques mobilisés : grands modèles de langue, représentations vectorielles (*embeddings*), génération augmentée par récupération (RAG) et modèles de vision-langage (VLM). Il analyse ensuite les approches existantes pour rendre un corpus multimodal interrogeable, les outils disponibles sur le marché et dans la littérature, leurs limites, et conclut par le positionnement précis de la solution développée.

1.2 Contexte : la documentation technique chez ALTEN

1.2.1 L'entreprise d'accueil

ALTEN est un groupe international d'ingénierie et de conseil en technologies (ETC), présent dans plus de trente pays. Le groupe accompagne les directions techniques et les directions des systèmes d'information de grands acteurs industriels (aéronautique, automobile, ferroviaire, énergie, télécommunications, sciences de la vie) sur l'ensemble du cycle de développement de leurs produits et systèmes. Les activités du groupe couvrent notamment l'ingénierie produit, les systèmes embarqués, les systèmes d'information industriels et, de plus en plus, la valorisation des données et l'intelligence artificielle.

1.2.2 Problématique métier

Par la nature même de ses activités, ALTEN manipule quotidiennement une documentation technique volumineuse et hétérogène : spécifications, manuels d'équipements, normes, articles de recherche, guides méthodologiques. Cette documentation présente trois caractéristiques qui rendent son exploitation difficile :

— **Hétérogénéité des formats** : PDF issus de chaînes d'édition variées, documents Word, classeurs Excel, avec des mises en page complexes (colonnes multiples, tableaux imbriqués, en-têtes et pieds de page parasites) ;
— **Information visuelle critique** : dans un manuel d'ingénierie électrique, par exemple, la réponse à « que représente ce symbole ? » se trouve dans un *schéma*, pas dans un paragraphe ; les diagrammes d'architecture des articles scientifiques concentrent souvent l'essentiel de la contribution ;
— **Coût d'accès élevé** : retrouver une information précise suppose de savoir dans quel document chercher, puis de le parcourir manuellement, un processus lent, frustrant et non scalable.

Le stage répond à cette problématique par une question d'ingénierie précise : *comment permettre à un ingénieur d'interroger en langage naturel un corpus de documents techniques, textes et figures compris, et d'obtenir une réponse fiable, vérifiable et illustrée ?*

1.3 Concepts et notions théoriques clés

1.3.1 Grands modèles de langue (LLM)

Les grands modèles de langue sont des réseaux de neurones fondés sur l'architecture *Transformer* [1], pré-entraînés sur d'immenses corpus à prédire le token suivant. Cette architecture, introduite par Vaswani et al. en 2017, remplace la récurrence par un mécanisme d'*auto-attention* qui met en relation tous les tokens d'une séquence en parallèle. Les modèles qui en découlent, de BERT [2], encodeur bidirectionnel pré-entraîné par MLM, aux modèles génératifs actuels (GPT-4o, Mistral, Llama, Gemma), présentent des capacités remarquables de compréhension et de génération de texte. Utilisés seuls, ils souffrent cependant de deux limites structurelles pour un usage documentaire : leur connaissance est figée à la date d'entraînement et n'inclut pas les documents internes d'une organisation, et ils peuvent produire des affirmations plausibles mais fausses (*hallucinations*).

1.3.2 Embeddings et recherche sémantique

Un *embedding* est une représentation d'un texte sous forme de vecteur dense de dimension fixe (1024 dimensions pour `mistral-embed`, utilisé dans ce projet), apprise de sorte que deux textes sémantiquement proches aient des vecteurs proches. La similarité entre une question et un passage se mesure alors par similarité cosinus (ou par distance euclidienne sur des vecteurs normalisés L2, ce qui revient au même classement). La *recherche sémantique* consiste à vectoriser le corpus une fois pour toutes, à stocker ces vecteurs dans une *base vectorielle*, puis, à chaque question, à rechercher les k vecteurs les plus proches du vecteur de la question. Contrairement à la recherche lexicale, elle retrouve des passages pertinents même sans mot commun avec la question (synonymes, reformulations, traductions).

1.3.3 Recherche lexicale : BM25

BM25 [4] est la fonction de classement lexical de référence, héritière de TF-IDF : elle score un document selon la fréquence des termes de la requête qu'il contient, pondérée par leur rareté dans le corpus et par la longueur du document. Ses forces sont exactement complémentaires de celles des embeddings : elle excelle sur les correspondances *exactes* (identifiants, références normatives, « Figure 1 », acronymes), là où deux libellés proches (« Figure 1 » et « Figure 2 ») produisent des embeddings quasi identiques que la similarité cosinus ne sait pas départager.

1.3.4 Retrieval-Augmented Generation (RAG)

Le RAG [3] articule ces briques en un pipeline en deux temps (figure 1.1) :

1. **Récupération (retrieval)** : la question de l'utilisateur est vectorisée, puis les passages du corpus les plus pertinents sont récupérés depuis l'index ;
2. **Génération augmentée** : ces passages sont injectés dans le prompt du LLM, avec l'instruction explicite de ne répondre *qu'à partir d'eux* et de citer ses sources.

Le LLM n'est donc plus une base de connaissances mais un *moteur de synthèse contraint* : la connaissance vit dans l'index documentaire, actualisable à tout moment, et chaque affirmation de la réponse est traçable vers sa source.

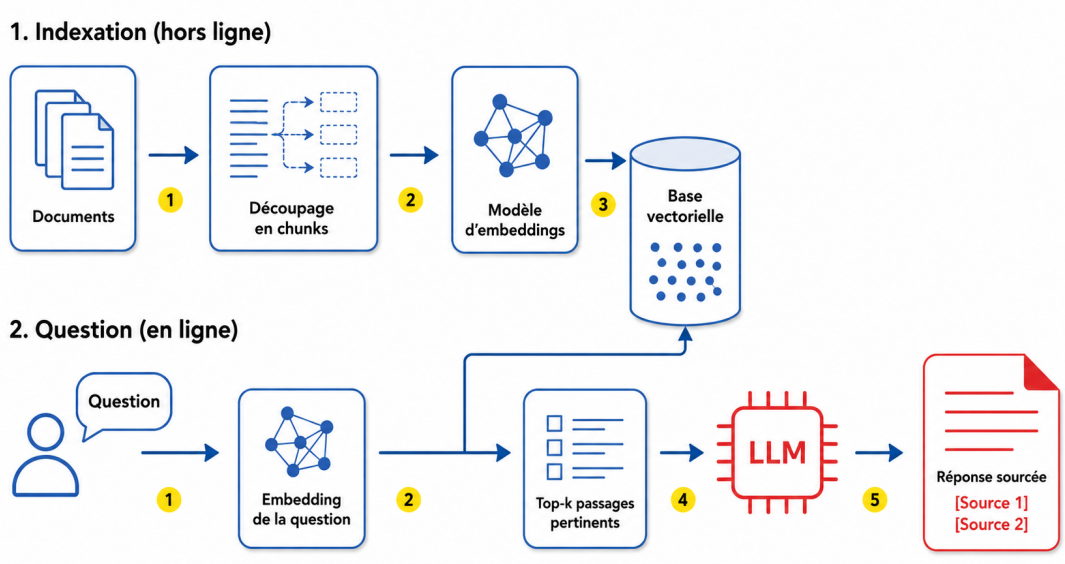


Figure 1.1 Principe général du paradigme RAG : indexation hors ligne du corpus, puis récupération et génération contrainte à chaque question.

1.3.5 Le découpage sémantique (chunking)

Les documents étant trop longs pour être vectorisés d'un bloc (et les fenêtres de contexte des modèles d'embeddings limitées, typiquement 512 tokens), le corpus doit être découpé en *chunks*.

La qualité de ce découpage conditionne directement celle du retrieval : un découpage naïf à taille fixe coupe les phrases et mélange les sections, tandis qu'un *découpage sémantique* respecte la structure logique du document (sections, paragraphes, tableaux) et rattache chaque chunk à son contexte hiérarchique (titres de sections englobantes). C'est l'approche du *HybridChunker* de Docling, utilisée ici.

1.3.6 Modèles de vision-langage (VLM)

Les modèles de vision-langage (GPT-4o vision, LLaVA, BLIP, Gemma-3) acceptent en entrée une image et un texte, et produisent du texte. Appliqués à une figure extraite d'un document, ils peuvent en produire une *description textuelle détaillée* : type de figure, composants, relations, textes et valeurs visibles. Cette capacité est la clé de voûte du RAG *multimodal* par description : elle projette le contenu visuel dans l'espace du texte, où il devient indexable et interrogeable avec les mêmes outils que le reste du corpus.

1.4 Approches existantes pour le RAG multimodal

La littérature et la pratique industrielle distinguent trois grandes stratégies pour rendre les images d'un corpus interrogeables (figure 1.2) :

1.4.1 OCR seul

L'approche historique applique une reconnaissance optique de caractères (Tesseract, Azure Document Intelligence) aux images pour en extraire le texte visible. Elle est efficace pour les documents scannés et les captures d'écran riches en texte, mais ne capture rien de la *sémantique visuelle* : un schéma électrique passé à l'OCR ne livre que quelques étiquettes isolées, pas la signification du circuit.

1.4.2 Captioning par VLM (description textuelle)

Chaque image est décrite par un VLM, et c'est la *description* qui est vectorisée et indexée aux côtés des chunks de texte, dans le même espace vectoriel. Cette approche offre un excellent rapport qualité/complexité : index unique et homogène, pipeline de retrieval inchangé, descriptions lisibles par l'utilisateur (utile pour l'affichage des sources). Sa limite : la qualité de l'index dépend de la qualité (et de l'orientation) des descriptions générées.

1.4.3 Embeddings multimodaux natifs

Des modèles comme CLIP [6] ou, plus récemment, ColPali [7] projettent images et textes dans un espace vectoriel *commun*, permettant de rechercher directement une image à partir d'une question textuelle, sans description intermédiaire. C'est l'approche la plus puissante sur le plan théorique, mais elle impose une infrastructure de retrieval spécifique (index séparé ou

multi-vecteurs, *late interaction* pour ColPali), des modèles plus lourds à héberger, et produit des représentations opaques pour l'utilisateur.

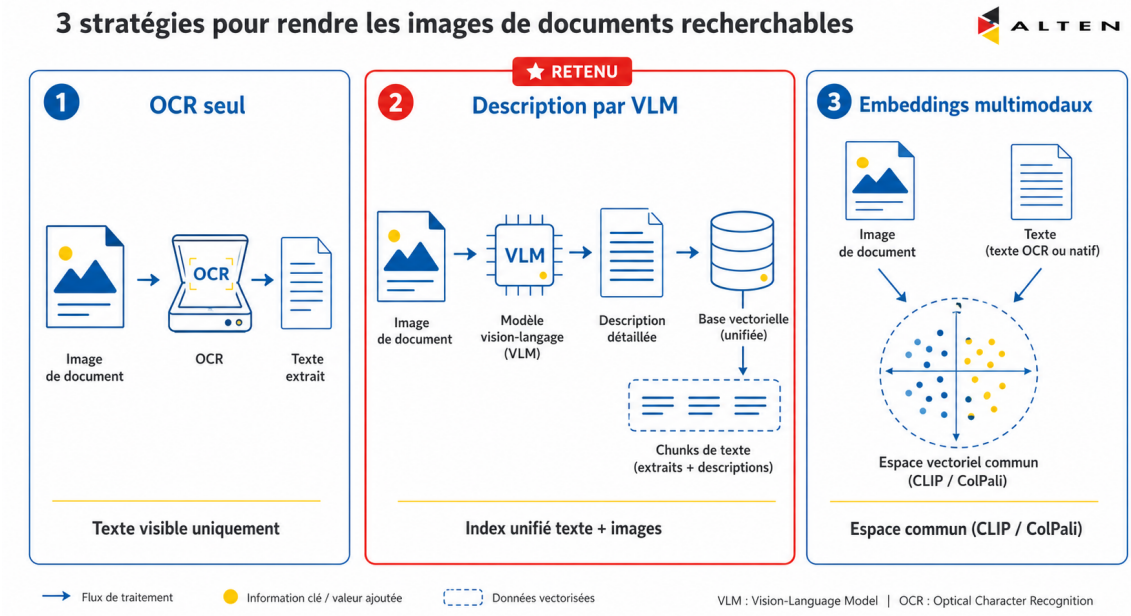


Figure 1.2 Les trois stratégies d'indexation des images d'un corpus : OCR seul, description par VLM, embeddings multimodaux natifs.

1.5 Outils et solutions existants

1.5.1 Cadres logiciels open source

LangChain et **LlamaIndex** sont les deux cadres dominants pour assembler des pipelines RAG : ils fournissent des abstractions prêtes à l'emploi (loaders, splitters, retrievers, chains). Leur généricité a un coût : couches d'abstraction épaisses, dépendances nombreuses, comportements parfois difficiles à auditer, un point sensible lorsqu'il faut diagnostiquer finement la qualité du retrieval.

1.5.2 Analyse documentaire

Pour l'extraction du contenu des PDF, l'écosystème s'étend des bibliothèques bas niveau (PyMuPDF, pdfplumber, pypdfium2) aux solutions d'analyse structurelle par apprentissage profond : **Docling** [9] (IBM Research, open source), qui combine un modèle de détection de mise en page et TableFormer pour la reconstruction des tableaux, et les services cloud managés comme **Azure Document Intelligence** ou Amazon Textract.

1.5.3 Bases vectorielles

Le marché offre un continuum allant des bibliothèques embarquées (**ChromaDB**, FAISS, LanceDB) aux serveurs dédiés (**Qdrant**, Weaviate, Milvus) et aux services managés (Pinecone). Les premières s'exécutent dans le processus applicatif sans infrastructure supplémentaire ; les seconds apportent réplication, sharding et très gros volumes.

1.5.4 Solutions intégrées du marché

Des produits clés en main adressent le même besoin : NotebookLM (Google), les assistants documentaires d'Azure AI Search « chat with your data », Glean, ou les GPTs personnalisés d'OpenAI. Ils offrent une mise en œuvre rapide mais imposent l'envoi du corpus chez un tiers, une personnalisation limitée du pipeline (chunking, retrieval, prompts) et des coûts de licence : trois points bloquants pour un démonstrateur interne maîtrisé de bout en bout.

1.6 Limites des solutions existantes

L'analyse de l'existant fait ressortir quatre limites au regard du besoin exprimé :

1. **Cécité visuelle des RAG standards** : les pipelines RAG « par défaut » (et la plupart des tutoriels/outils grand public) n'indexent que le texte ; les figures, porteuses d'une part essentielle de l'information technique, sont ignorées ;
2. **Retrieval purement dense** : la recherche par embeddings seuls échoue sur les requêtes exactes (références de figures, acronymes, identifiants), fréquentes dans un contexte d'ingénierie ;
3. **Extraction PDF simpliste** : les loaders textuels basiques détruisent la structure (tableaux aplatis, colonnes mélangées, légendes orphelines), dégradant mécaniquement la qualité des chunks puis des réponses ;
4. **Inadéquation aux environnements contraints** : les solutions supposant l'installation de services (Docker, serveurs de bases de données, OCR système) ou l'accès libre à Internet sont inapplicables sur un poste d'entreprise verrouillé derrière un proxy d'inspection TLS.

1.7 Apport de la solution proposée

La solution développée répond point par point à ces limites :

Apports de la solution

— **Indexation multimodale unifiée** : les figures sont extraites par Docling, décrites par un VLM (Gemma-3 27B) avec un prompt d'ancrage contextuel, et leurs descriptions sont indexées *dans le même espace vectoriel* que le texte : une question peut donc être résolue indifféremment par un paragraphe ou par une figure, laquelle est alors affichée à l'utilisateur ;

— **Recherche hybride dense + BM25 fusionnée par RRF** : la complémentarité sémantique/lexicale est exploitée sans calibration de scores, par fusion de rangs réciproques [5] ;
— **Analyse structurelle de qualité** : Docling préserve la structure logique des documents (sections, tableaux, légendes de figures), et le découpage *HybridChunker* produit des chunks contextualisés respectant la limite de tokens du modèle d'embeddings ;
— **Architecture sans dépendance système** : SQLite, ChromaDB persistant et FakeRedis s'exécutent dans le processus Python ; les modèles lourds sont consommés via des API cloud accessibles à travers le proxy d'entreprise ; le tout fonctionne sur un poste verrouillé et se déploie tel quel sur une VM OCI ;
— **Qualité mesurée, pas supposée** : le système est évalué quantitativement avec RAGAS [8] et un juge LLM indépendant du système évalué.

1.8 Positionnement du projet

Le tableau 1.1 synthétise le positionnement de la solution par rapport aux familles d'alternatives. Le projet assume des choix de *sobriété architecturale* adaptés à son cadre (un démonstrateur de huit semaines destiné à un déploiement OCI léger) : pipeline RAG assemblé directement sur les SDK des fournisseurs (sans LangChain), stockage embarqué, approche multimodale par description VLM plutôt que par embeddings multimodaux natifs. Conformément au cahier des charges, les architectures multi-agents, le RAG auto-correctif et la gestion fine des droits sont explicitement hors périmètre et traités comme perspectives.

Table 1.1 Positionnement de la solution développée par rapport à l'existant.

Dimension	Alternatives courantes	Choix du projet
Analyse PDF	Loaders textuels naïfs ; OCR cloud payant	Docling (layout + TableFormer), modèles locaux pré-téléchargés
Images	Ignorées, ou OCR seul	Captioning VLM contextualisé + indexation unifiée
Retrieval	Dense seul	Hybride dense + BM25, fusion RRF
Base vectorielle	Serveur dédié (Qdrant, Milvus)	ChromaDB embarqué persistant
Orchestration	LangChain / LlamaIndex	Pipeline explicite sur SDK natifs (auditable)
Génération	GPT-4o (OpenAI)	Mistral `mistral-small-latest` (accessible via le proxy, coût maîtrisé)
Évaluation	Tests manuels	RAGAS + juge LLM indépendant (Llama 3.3 70B)

1.9 Conclusion

Ce chapitre a établi le socle conceptuel du projet (LLM, embeddings, RAG, VLM, recherche hybride) et montré que les solutions existantes laissent un espace précis : celui d'un assistant docu-

mentaire multimodal, maîtrisé de bout en bout, fonctionnant dans un environnement d'entreprise contraint. Le chapitre suivant traduit ce positionnement en une architecture logicielle concrète : besoins, pipeline, composants, modélisation et justification détaillée des choix technologiques.

2 Chapitre

Analyse, conception et architecture de la solution

2.1 Introduction

Ce chapitre traduit le positionnement établi au chapitre précédent en une conception logicielle complète. Après une présentation générale du système et la formalisation des besoins fonctionnels et non fonctionnels, il décrit le pipeline de traitement, l'architecture globale à trois niveaux (frontend React, backend FastAPI, couche de persistance), le détail des composants du backend, les flux entre composants, la modélisation UML, le modèle de données, et s'achève sur la justification argumentée de chaque choix technologique.

2.2 Présentation générale du projet

Le système développé, baptisé *Assistant RAG Multimodal*, est une application web complète permettant à un utilisateur authentifié de :

- téléverser des documents techniques (PDF, Word, Excel) et les organiser en *groupes documentaires* (par exemple « Research Paper », « Technical Manual ») ;
- suivre en temps réel les étapes d'ingestion de chaque document (analyse, découpage, description des figures, vectorisation, indexation) ;
- consulter chaque document indexé : rendu Markdown du contenu extrait, galerie des figures avec leurs descriptions générées ;
- poser des questions en langage naturel dans une interface conversationnelle multi-conversations, avec choix du périmètre de recherche (tout le corpus, sélection manuelle de documents, ou routage automatique par LLM) ;
- obtenir des réponses *sourcées* : chaque affirmation cite ses sources `[Source N]`, et lorsque la source est une figure, l'image elle-même est affichée sous la réponse.

Le corpus de démonstration comprend des articles de recherche fondateurs en apprentissage profond (*Attention Is All You Need*, BERT, ResNet, ViT, YOLO) et des manuels d'ingénierie électrique (symboles de schémas, lecture de plans), deux familles choisies pour leur richesse en

figures techniques.

2.3 Besoins fonctionnels

Table 2.1 Synthèse des besoins fonctionnels.

Réf.	Besoin
BF-01	Téléverser des documents PDF, DOCX et XLSX, avec affectation à un groupe documentaire
BF-02	Ingestion automatique : analyse structurelle, découpage sémantique, extraction et description des figures, vectorisation, indexation
BF-03	Suivi de l'état d'ingestion (étape courante, succès, erreur) document par document
BF-04	Gestion des documents : consultation détaillée (Markdown + figures), renommage, changement de groupe, duplication, suppression, téléchargement du fichier source
BF-05	Gestion des groupes documentaires : création, renommage, suppression (avec repli des documents vers le groupe par défaut)
BF-06	Poser une question en langage naturel et recevoir une réponse générée, ancrée dans le corpus, avec citations `[Source N]`
BF-07	Restitution multimodale : affichage des figures sources sous la réponse lorsqu'elles sont citées
BF-08	Périmètre de recherche : corpus entier, documents sélectionnés manuellement, ou routage automatique de la question vers les groupes pertinents par LLM
BF-09	Historique de conversations par utilisateur : création, consultation, reprise, suppression
BF-10	Authentification des utilisateurs (inscription, connexion) et cloisonnement des conversations par utilisateur

2.4 Besoins non fonctionnels

Table 2.2 Synthèse des besoins non fonctionnels.

Réf.	Besoin
BNF-01	**Fiabilité des réponses** : le LLM ne doit répondre qu'à partir des sources récupérées et déclarer explicitement l'absence de réponse plutôt que d'inventer
BNF-02	**Traçabilité** : toute réponse doit être vérifiable (document, page, extrait ou figure)
BNF-03	**Compatibilité environnement contraint** : fonctionnement sans droits administrateur, sans service système à installer, à travers un proxy d'inspection TLS
BNF-04	**Robustesse de l'ingestion** : un document en erreur ne doit ni bloquer la file ni corrompre l'index ; les erreurs sont enregistrées et affichées
BNF-05	**Stabilité sous charge locale** : les téléversements en rafale ne doivent pas saturer la machine ni provoquer de contention d'écriture (sérialisation des ingestions)
BNF-06	**Sécurité** : vérification cryptographique des jetons de session (RS256/JWKS), contrôle de propriété des conversations, CORS restrictif, validation des types de fichiers
BNF-07	**Portabilité cloud** : déploiement cible sur OCI sans refonte (stockage embarqué, cache commutable vers un Redis managé par simple variable d'environnement)
BNF-08	**Qualité mesurable** : le système doit être évaluable quantitativement (métriques RAGAS reproductibles)
BNF-09	**Coût maîtrisé** : recours aux offres gratuites/économiques des fournisseurs de modèles (Mistral, DeepInfra via Hugging Face, Groq pour l'évaluation)

2.5 Principe général du pipeline

Le système s'organise en deux chaînes asynchrones l'une de l'autre : une chaîne d'**ingestion** (hors ligne, déclenchée par chaque téléversement) et une chaîne de **restitution** (en ligne, déclenchée par chaque question). La figure 2.1 en donne la vue d'ensemble.

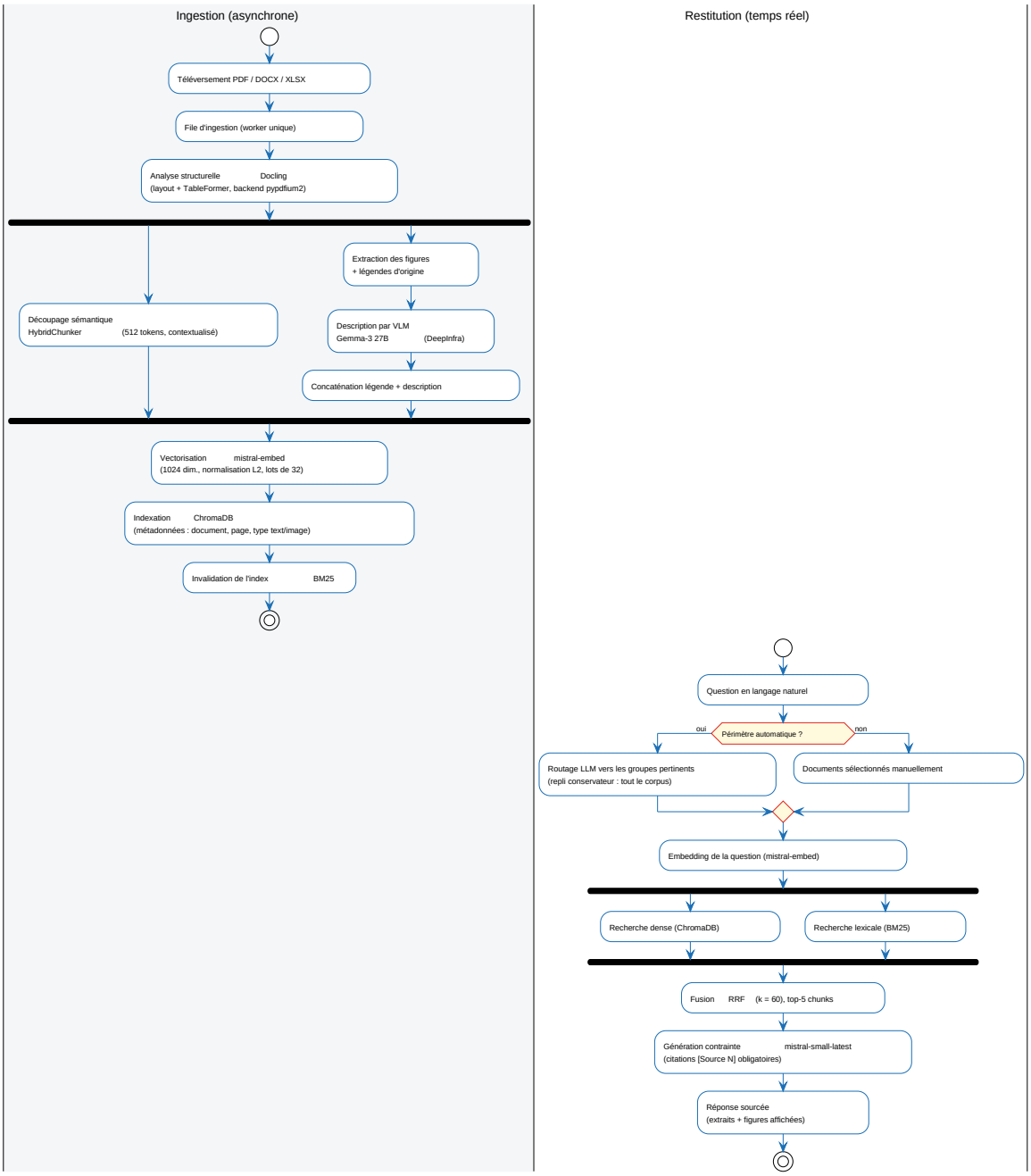


Figure 2.1 Pipeline fonctionnel complet : chaîne d'ingestion (à gauche) et chaîne de restitution (à droite), articulées autour de l'index unifié ChromaDB + BM25.

2.5.1 Chaîne d'ingestion

1. **Analyse structurelle (Docling)** : le document est converti en une représentation structurée (sections, paragraphes, tableaux, figures avec leurs légendes et numéros de pages). Le backend d'analyse PDF retenu est `pypdfium2`, plus robuste en mémoire que le backend par défaut ;
2. **Export Markdown** : le contenu structuré est exporté en Markdown et conservé pour la consultation dans l'interface ;
3. **Découpage sémantique** : le *HybridChunker* de Docling produit des chunks respectant la structure logique et la limite de 512 tokens, chacun *contextualisé* (préfixé par ses titres de sections) et rattaché à sa page ;
4. **Extraction et description des figures** : chaque image est sauvegardée sur disque puis décrite par le VLM Gemma-3 27B ; la légende d'origine trouvée par Docling (« Figure 1 : ... ») est préfixée à la description générée, pour que la recherche lexicale sur « Figure 1 » ait une prise ;
5. **Vectorisation** : chunks de texte et descriptions de figures sont plongés dans le même espace vectoriel par `mistral-embed` (1024 dimensions, vecteurs normalisés L2), par lots de 32 ;
6. **Indexation** : les vecteurs, textes et métadonnées (document, page, type `text`/`image`, chemin de l'image) sont insérés dans la collection ChromaDB ; l'index BM25 en mémoire est marqué à reconstruire.

2.5.2 Chaîne de restitution

1. **Périmètre** : si l'utilisateur n'a pas sélectionné de documents et a activé le périmètre automatique, un appel LLM route la question vers les groupes documentaires plausiblement pertinents (avec repli conservateur sur tout le corpus) ;
2. **Recherche hybride** : la question est vectorisée, puis deux recherches sont lancées avec sur-échantillonnage ($3 \times k$) : dense (ChromaDB) et lexicale (BM25) ; leurs classements sont fusionnés par RRF et les $k = 5$ meilleurs chunks sont retenus ;
3. **Génération contrainte** : les chunks numérotés (avec document et page) sont injectés dans le prompt de `mistral-small-latest`, dont les instructions imposent de ne répondre qu'à partir des sources, de citer `[Source N]` et de signaler l'absence de réponse ;
4. **Restitution sourcée** : la réponse est renvoyée avec la liste structurée des sources ; celles de type figure portent l'URL de l'image, affichée par le frontend.

2.6 Architecture globale du système

L'architecture (figure 2.2) suit un découpage classique en trois niveaux, enrichi d'une couche de services d'IA externes :

— **Frontend** : SPA React 18 + TypeScript servie par Vite, intégrant le SDK Clerk pour l'authentification ; elle dialogue avec le backend en JSON/HTTP en joignant le jeton de session à chaque requête ;
— **Backend** : application FastAPI exposant trois familles de routes (`/api/documents`, `/api/groups`, `/api/chat` + `/api/conversations`), un serveur de fichiers statiques pour les images extraites, et un *worker* d'ingestion sur file en arrière-plan ;
— **Persistance** : SQLite (métadonnées des documents, groupes, conversations, messages), ChromaDB persistant (vecteurs + textes + métadonnées), système de fichiers (fichiers sources, exports Markdown, images extraites), cache Redis commutable (FakeRedis en développement, OCI Cache en production) ;
— **Services d'IA externes** : API Mistral (embeddings `mistral-embed` et génération `mistral-small-latest`), API Hugging Face Inference / DeepInfra (captioning Gemma-3 27B), Clerk (JWKS de vérification des jetons), Groq (juge Llama 3.3 70B, pour l'évaluation uniquement).

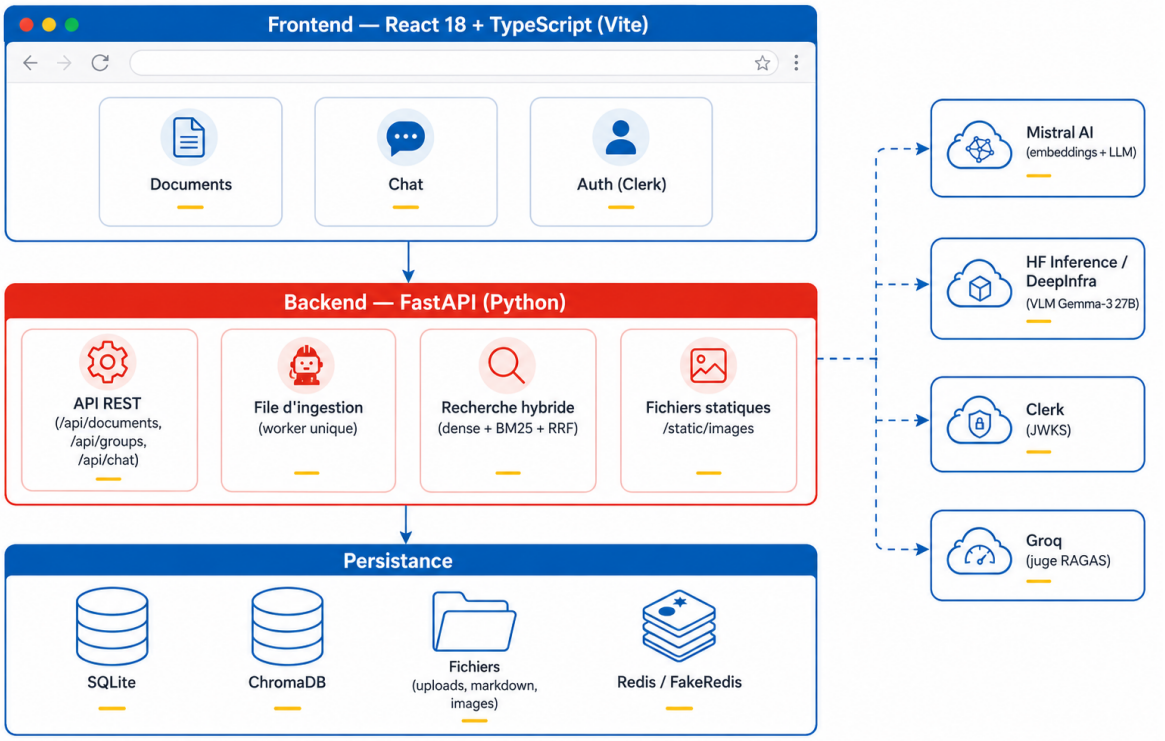


Figure 2.2 Architecture globale : SPA React, API FastAPI, couche de persistance embarquée et services d'IA externes.

2.7 Architecture du backend

Le backend suit une organisation en couches nettes (routes, services, stockage), sans framework d'orchestration intermédiaire (figure 2.3) :

`api/` les routeurs FastAPI (`routes_documents`, `routes_groups`, `routes_chat`) : validation des entrées (schémas Pydantic), contrôle d'accès, orchestration des services ; aucune logique

métier ;

`services/` le cœur métier, un module par responsabilité : **`docling_parser`** (analyse, découpage, extraction d'images), **`ingestion`** et **`ingestion_queue`** (pipeline et sérialisation), **`hf_client`** (embeddings multi-backends et captioning VLM), **`mistral_client`** (embeddings, génération, routage de groupes), **`vector_store`** (ChromaDB, BM25, fusion RRF), **`clerk_auth`** (vérification JWT), **`cache`** (cache JSON fail-open), et les dépôts **`documents_repo`**, **`groups_repo`**, **`conversations_repo`** (accès SQLite) ;

`storage/` la résolution centralisée des chemins de données (**`uploads/`**, **`markdown/`**, **`images/`**, **`chroma/`**, **`app.db`**) ;

`config.py` la configuration typée **`pydantic-settings`**, chargée depuis **`.env`** : clés d'API, choix du backend d'embeddings, chemins des modèles hors ligne, URL Redis, origines CORS.

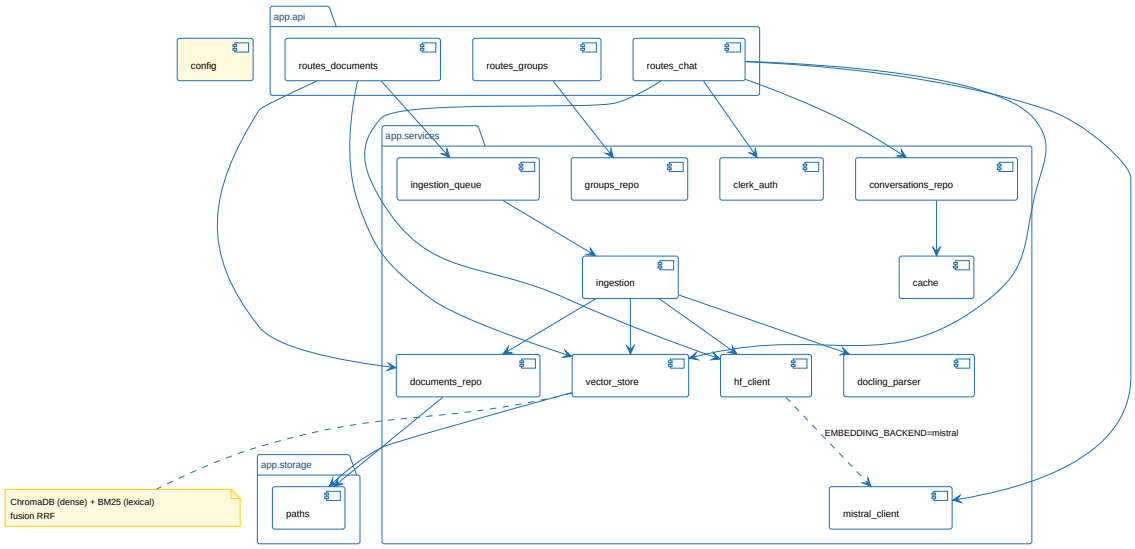


Figure 2.3 Diagramme de paquetages du backend : dépendances entre routes, services et stockage.

Un choix structurant : la file d'ingestion à worker unique

L'ingestion (Docling + VLM + embeddings) est lourde en CPU et en mémoire. Le traitement concurrent des téléversements en rafale saturait la machine et provoquait des contentions d'écriture SQLite/ChromaDB allant jusqu'à l'échec de documents. L'architecture retenue, une file en mémoire consommée par un *unique* thread worker démarré avec l'application, garantit un traitement strictement séquentiel, dans l'ordre d'arrivée, quelle que soit la rafale. L'API reste, elle, parfaitement réactive : le téléversement répond immédiatement et l'interface suit la progression par interrogation périodique.

2.8 Architecture du frontend

Le frontend est structuré par domaines fonctionnels : **`components/auth`** (pages et garde d'authentification Clerk), **`components/documents`** (vue corpus, carte et détail de document,

dropzone de téléversement, galerie de figures), `components/chat` (conversation, bulles de messages avec rendu Markdown, cartes de sources, visionneuse de figures, sélecteur de périmètre documentaire), `components/layout` et `components/ui` (coquille applicative, navigation, composants réutilisables, bascule de thème clair/sombre). Un client API central (`api/client.ts`) joint le jeton Clerk à chaque appel ; le hook `useChat` encapsule l'état conversationnel ; le routage est assuré par React Router avec routes protégées.

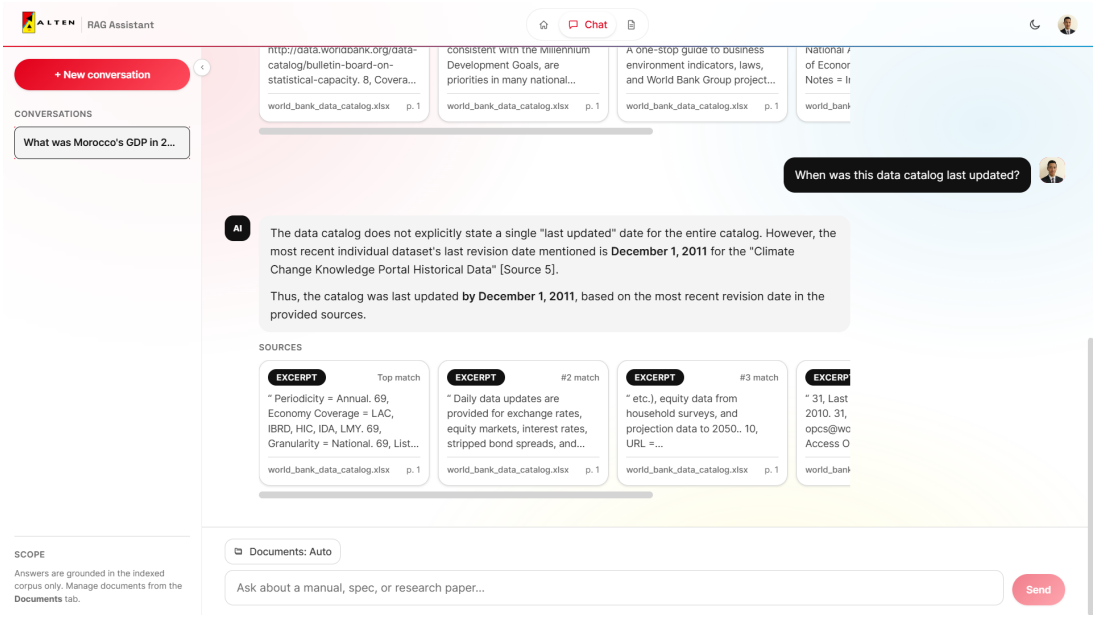


Figure 2.4 Interface conversationnelle : réponse sourcée avec citations et figure affichée sous la réponse.

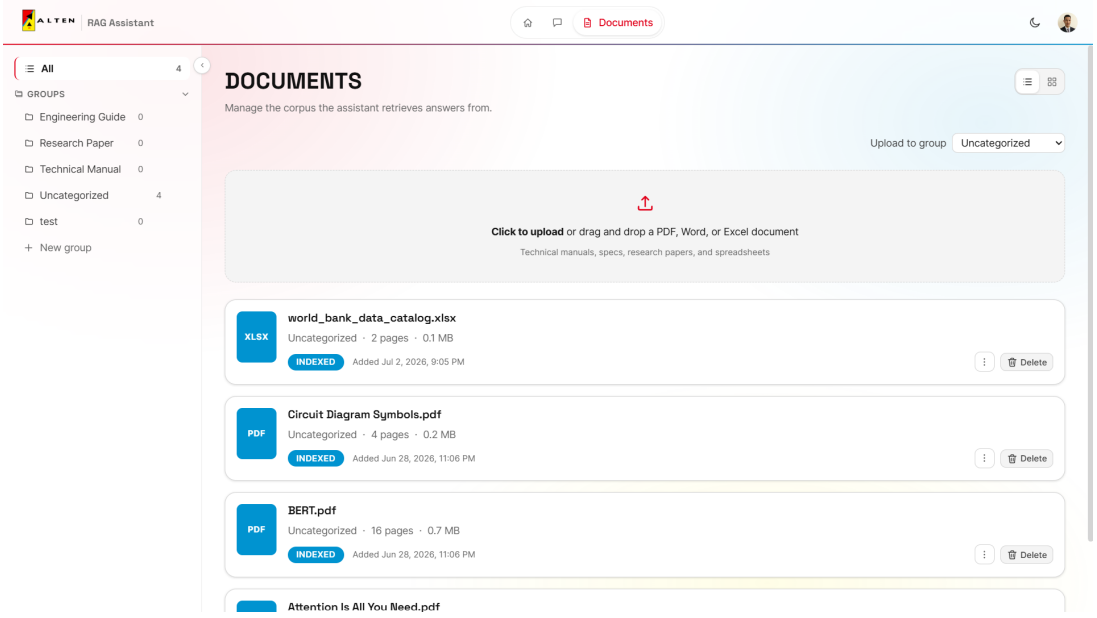


Figure 2.5 Vue de gestion du corpus : groupes documentaires, état d'ingestion en temps réel et actions par document.

2.9 Communication et flux entre composants

Tous les échanges frontend–backend sont des appels REST JSON authentifiés par jeton Bearer, hormis les images servies statiquement (`/static/images/...`). Les deux flux majeurs sont modélisés en séquence : le flux d'ingestion (figure 2.6) illustre le découplage entre la requête de téléversement (synchrone, immédiate) et le traitement (asynchrone, séquentiel) ; le flux de question (figure 2.7) montre l'enchaînement routage automatique – recherche hybride – génération – restitution sourcée.

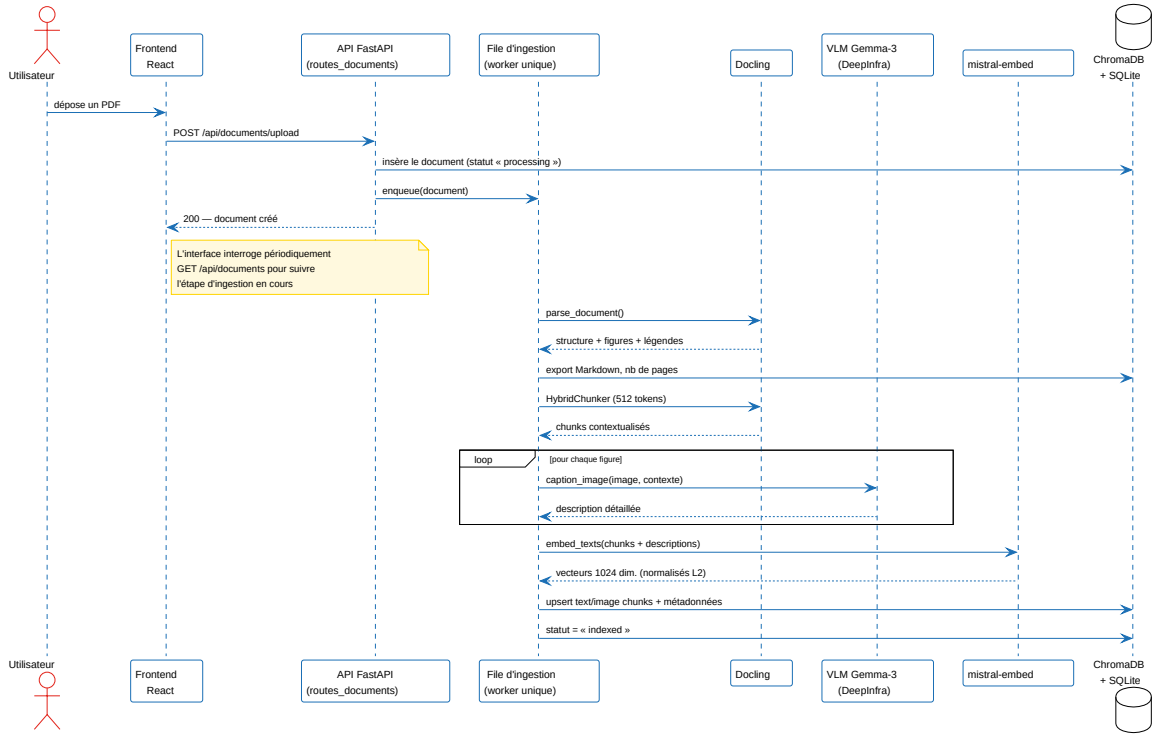


Figure 2.6 Diagramme de séquence du flux d'ingestion d'un document.

2.10 Modélisation UML

Outre les diagrammes de séquence ci-dessus, deux vues complètent la modélisation. Le diagramme de cas d'utilisation (figure 2.8) recense les interactions de l'utilisateur authentifié avec le système ; le diagramme de classes du modèle de données (figure 2.9) est présenté en section suivante.

2.11 Modèle de données

La persistance repose sur une séparation stricte entre *métadonnées relationnelles* (SQLite) et *contenu vectorisé* (ChromaDB), reliés par l'identifiant de document (UUID) :

— **SQLite**, quatre tables : `documents` (identité, groupe, état d'ingestion, étape courante, message d'erreur, chemin du fichier, lien `source_document_id` pour les duplicatas),

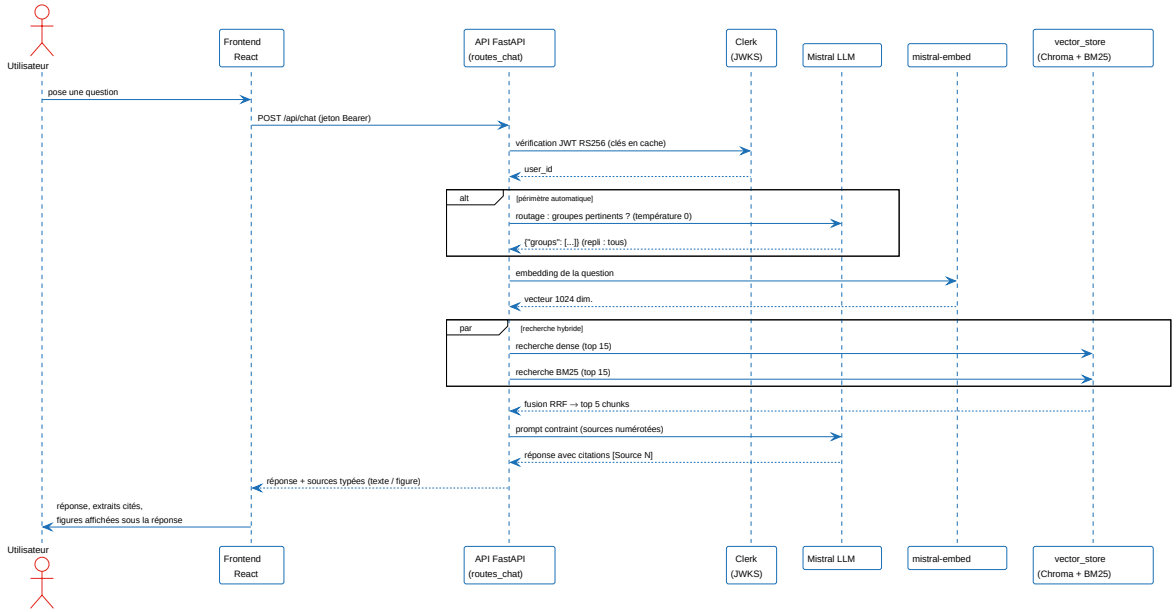


Figure 2.7 Diagramme de séquence du flux de question–réponse sourcée.

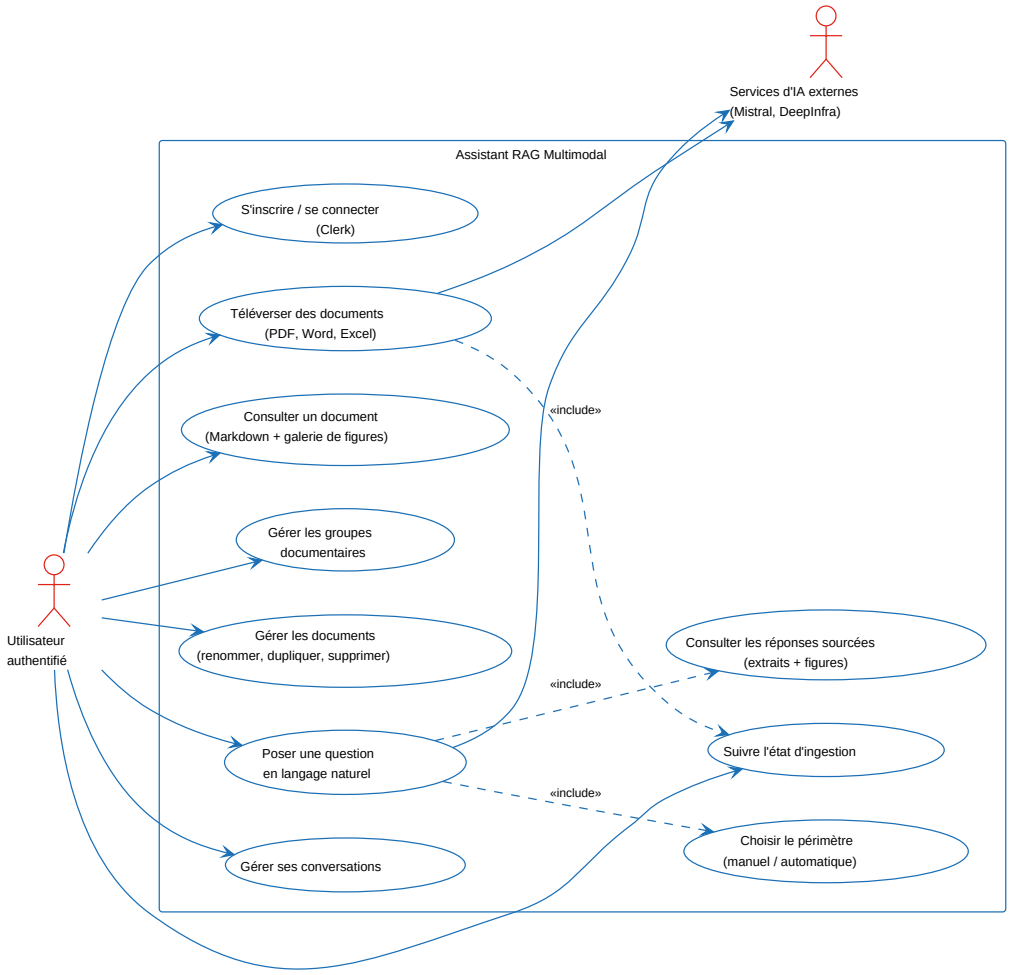


Figure 2.8 Diagramme de cas d'utilisation de l'assistant RAG multimodal.

`groups` (groupes documentaires nommés, avec groupe par défaut non supprimable), `conversations` (rattachées à un `user_id` Clerk) et `messages` (rôle, contenu, sources sérialisées en JSON, horodatage) ;

— **ChromaDB**, une collection unique `documents` où chaque entrée porte : le vecteur (1024 dim.), le texte (chunk ou description de figure) et les métadonnées `{document_id, document_name, page, chunk_type (text|image), image_path}` ; c'est ce champ `chunk_type` qui matérialise l'indexation multimodale unifiée ;

— **Système de fichiers** : `uploads/` (originaux), `markdown/` (exports), `images/<document_id>/` (figures extraites).

Un point de conception notable : la *duplication* d'un document ne copie ni fichier, ni vecteurs, ni images : le duplicata référence le document racine via `source_document_id`, et la consultation comme la suppression résolvent ce lien (seule la suppression du document racine détruit le contenu partagé).

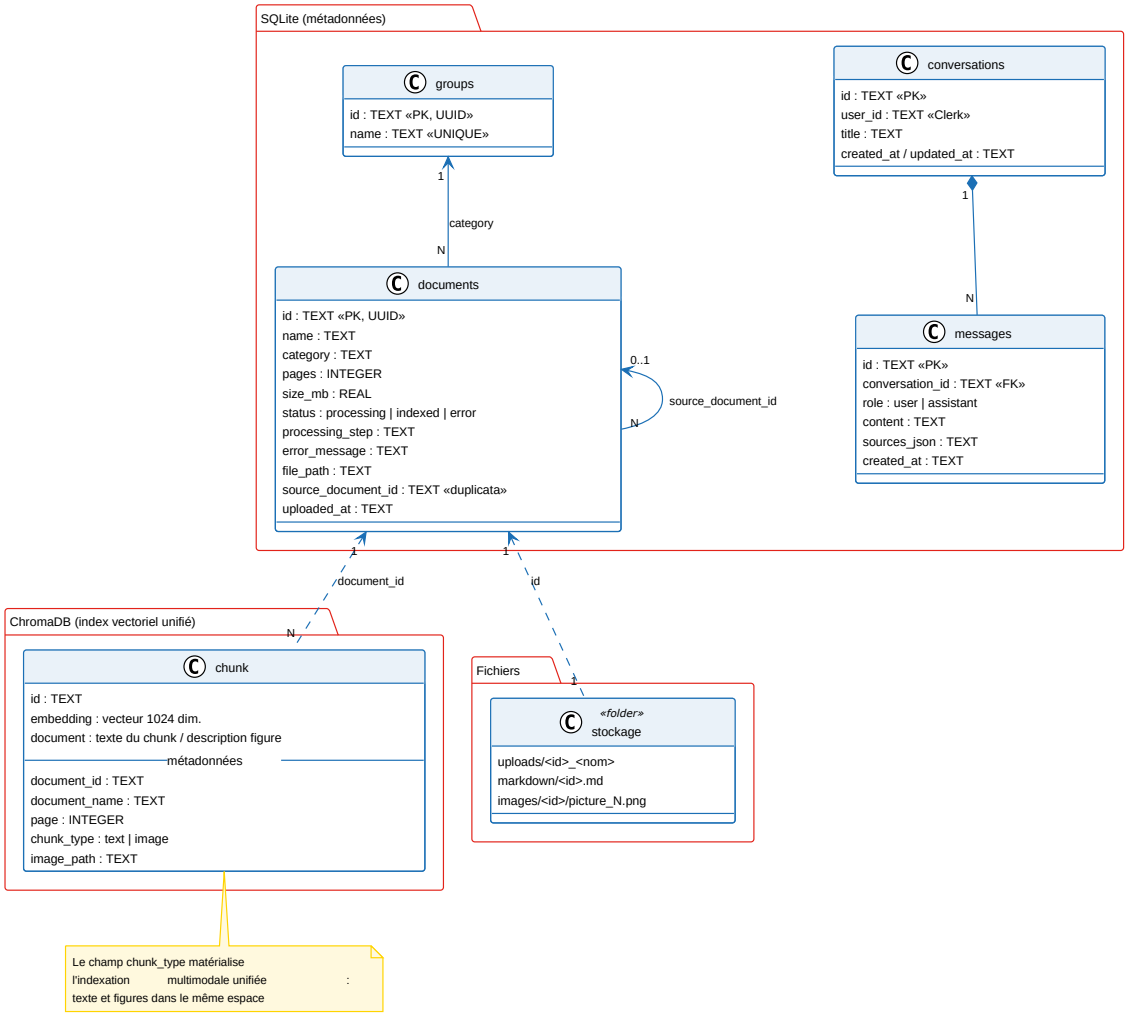


Figure 2.9 Modèle de données : tables SQLite, collection ChromaDB et stockage fichiers, reliés par l'identifiant de document.

2.12 Justification des choix technologiques

Table 2.3 Choix technologiques et justifications.

Besoin	Choix	Justification
Analyse documentaire	Docling	Open source, détection de mise en page et TableFormer par apprentissage profond, extraction native des figures avec légendes, chunker sémantique intégré ; modèles téléchargeables hors ligne (indispensable derrière le proxy)
Backend PDF	pypdfium2	Le backend Docling par défaut provoquait des `std::bad_alloc` sur des PDF à images volumineuses ; pypdfium2 est plus robuste en mémoire
Embeddings	`mistral-embed`	Seule API d'embeddings de qualité *accessible à travers Zscaler* (Hugging Face bloqué) ; 1024 dim., multilingue, batch natif ; backends alternatifs (HF Inference, local transformers) conservés et commutables par configuration
Captioning VLM	Gemma-3 27B (DeepInfra)	Excellent rapport qualité/coût pour la description de figures techniques ; accessible via l'API unifiée Hugging Face Inference Providers
Base vectorielle	ChromaDB	Embarquée et persistante : zéro serveur à installer (PC verrouillé), portage OCI trivial ; API de filtrage par métadonnées suffisante au besoin
Retrieval lexical	`rank_bm25` + RRF	Bibliothèque pure Python ; RRF fusionne les classements sans calibration de scores (constante $k = 60$ de Cormack et al.)
LLM de génération	`mistral-small-latest`	Qualité suffisante pour la synthèse contrainte, latence et coût faibles, API stable et accessible
API backend	FastAPI	Validation Pydantic, documentation OpenAPI automatique, montage de fichiers statiques, écosystème Python (Docling, ChromaDB)
Frontend	React 18 + Vite + TypeScript	SPA réactive, typage de bout en bout avec les schémas Pydantic, rechargement instantané
Authentification	Clerk	Authentification déléguée complète (UI incluse) sans stockage de mots de passe ; vérification backend purement cryptographique (JWKS), sans appel réseau par requête
Cache	Redis / FakeRedis	Même code en dev (FakeRedis en mémoire, aucun serveur) et en prod (OCI Cache), commutation par variable d'environnement ; conception *fail-open*
Évaluation	RAGAS + Groq (Llama 3.3 70B)	Métriques standardisées ; juge indépendant du système évalué pour éviter le biais d'auto-évaluation

Deux arbitrages méritent un développement :

Sans LangChain. Le pipeline est assemblé directement sur les SDK natifs (`mistralai`, `huggingface_hub`, `chromadb`). Pour un pipeline de cette taille, les abstractions de LangChain auraient ajouté une épaisseur difficile à auditer sans rien apporter : chaque étape (chunking, embeddings, fusion, prompt) est ici un module Python explicite de quelques dizaines de lignes, directement instrumentable, un atout décisif lors du diagnostic de qualité du retrieval.

Captioning plutôt qu'embeddings multimodaux. L'approche par description VLM (section 1.4) a été préférée à CLIP/ColPali : elle conserve un index unique et un pipeline de

retrieval inchangé, produit des descriptions réutilisées telles quelles dans l'interface (galerie de figures, cartes de sources), et ne requiert aucun modèle auto-hébergé, rédhibitoire sur le poste de développement.

2.13 Conclusion

L'architecture conçue répond aux besoins formalisés par un système à trois niveaux, sobre et entièrement auditable : une chaîne d'ingestion asynchrone sérialisée qui transforme des documents hétérogènes en un index multimodal unifié, et une chaîne de restitution qui combine routage LLM, recherche hybride RRF et génération contrainte citée. Chaque choix technologique a été arbitré au regard du triptyque qualité / contraintes d'environnement / portabilité OCI. Le chapitre suivant décrit la mise en œuvre concrète de cette architecture et sa validation expérimentale.

3 Chapitre

Réalisation, implémentation et validation

3.1 Introduction

Ce dernier chapitre décrit la mise en œuvre effective de l'architecture conçue au chapitre 2. Il présente l'environnement de développement (dont les contraintes ont pesé sur la réalisation autant que le cahier des charges lui-même), détaille l'implémentation des composants clés et l'intégration des services tiers, expose les mécanismes de sécurité applicative, puis la démarche de validation : évaluation quantitative RAGAS, scénario de démonstration, résultats obtenus, difficultés rencontrées et limites de la solution.

3.2 Environnement de développement

3.2.1 Poste de travail et contraintes

Le développement s'est déroulé sur un poste professionnel ALTEN sous Windows 11, *verrouillé* : pas de droits d'administration, pas d'installation d'applications ou de services système (Docker, serveurs de bases de données, Tesseract...), et un agent **Zscaler Client Connector** interceptant l'intégralité du trafic HTTPS sortant par inspection TLS, quel que soit le réseau utilisé, l'agent étant applicatif et non réseau.

Conséquences concrètes des contraintes

- Toute dépendance devait être installable *sans droits administrateur* : environnement virtuel Python (`venv`), paquets `pip` et `npm` locaux au projet ;
- Aucun service ne pouvait être installé : d'où SQLite, ChromaDB embarqué et FakeRedis en processus ;
- L'inspection TLS de Zscaler re-signe le trafic HTTPS avec son propre certificat racine, reconnu par Windows mais pas par la liste `certifi` embarquée de Python : toutes les requêtes HTTPS Python échouaient en `CERTIFICATE_VERIFY_FAILED` alors que le navigateur fonctionnait ;
- Certains domaines étaient bloqués : le CDN de stockage de Hugging Face (téléchargement de modèles) et son API d'inférence, ce qui a imposé des stratégies de

contournement décrites plus loin.

3.2.2 Outillage

L'environnement s'appuie sur : Python 3.12 en environnement virtuel, Node.js/npm pour le frontend (Vite assurant serveur de développement et rechargement à chaud), Git/GitHub pour le versionnement, Visual Studio Code assisté d'outils d'IA de développement, et des notebooks Jupyter pour les expérimentations préalables (validation du parsing Docling sur *Attention Is All You Need*, comparaison de VLM pour le captioning, campagne d'évaluation RAGAS). L'API est documentée automatiquement par le Swagger UI de FastAPI (`/docs`), qui a servi de banc de test permanent.

3.3 Implémentation des composants principaux

3.3.1 Analyse documentaire et découpage (`docling_parser`)

Le convertisseur Docling est construit une seule fois (mise en cache par `lru_cache`) avec des options dédiées : génération des images de figures activée, OCR désactivé (hors périmètre de cette itération), backend PDF `pypdfium2` et, lorsque la variable `DOCLING_ARTIFACTS_PATH` est renseignée, chargement des modèles de layout et de reconstruction de tableaux depuis un dossier local plutôt que depuis Hugging Face : le mécanisme qui a permis de fonctionner malgré le blocage du CDN.

Le découpage utilise le *HybridChunker* de `docling-core`, paramétré par le tokenizer du modèle d'embeddings (512 tokens max) ; chaque chunk est *contextualisé* (préfixé des titres de sections englobantes) et rattaché à son numéro de page pour la citation. Un découpage de repli à fenêtre glissante (1500 caractères, chevauchement de 200) prend le relais si le chunker sémantique échoue sur un document atypique.

3.3.2 Description des figures par VLM (`hf_client`)

Chaque figure extraite est envoyée au VLM **Gemma-3 27B** (servi par DeepInfra via l'API unifiée Hugging Face Inference Providers) sous forme d'URL de données base64, avec un prompt d'ancrage soigneusement construit : nom du document et légende d'origine de la figure fournis comme contexte, consigne de décrire type, composants, relations, textes et valeurs visibles, interdiction des préambules (« Voici une description... ») et du *hedging* sur les faits déjà donnés. La description finale indexée est la concaténation *légende d'origine + description VLM* : la légende (« Figure 1 : The Transformer architecture ») donne une prise à la recherche lexicale exacte, la description VLM la profondeur sémantique.

Les appels aux API d'inférence sont protégés par une politique de nouvelle tentative sélective (bibliothèque `tenacity`) : trois essais à attente exponentielle aléatoire sur les erreurs 5xx

transitoires (démarrages à froid des fournisseurs serverless), aucune tentative sur les 4xx (erreur d'authentification ou de charge utile, qu'un nouvel essai ne réparera pas).

3.3.3 Embeddings multi-backends

Le module d'embeddings illustre l'adaptation aux contraintes réseau : trois backends commutables par la seule variable `EMBEDDING_BACKEND`, derrière une interface unique `embed_texts()` :

- `hf_inference` : `multilingual-e5-large` via l'API Hugging Face, le choix initial, devenu inutilisable lorsque le domaine a été bloqué ;
- `local` : les mêmes poids e5-large exécutés sur CPU via `transformers`, chargés depuis un dossier local en `local_files_only` (jamais de réseau), avec *mean pooling* masqué et normalisation L2 reproduisant exactement la recette du serveur d'inférence HF, garantissant des vecteurs compatibles avec l'index existant ;
- `mistral` : `mistral-embed` via l'API Mistral, accessible à travers Zscaler, le backend actif. Les vecteurs (1024 dim.) sont normalisés L2 pour que la distance de ChromaDB ordonne comme une similarité cosinus, et envoyés par lots de 32 (une requête pour plusieurs dizaines de textes, compatible avec la limite de débit du palier gratuit).

Un piège d'ingénierie : deux espaces vectoriels de même dimension

`mistral-embed` et e5-large produisent tous deux des vecteurs de 1024 dimensions mais dans des espaces *incompatibles*. ChromaDB ne signale donc aucune erreur si l'on change de backend sans réindexer : il renvoie silencieusement des similarités absurdes. La bascule vers Mistral a ainsi nécessité la purge de la collection et la réingestion complète du corpus, et le risque est explicitement documenté dans la configuration.

3.3.4 Recherche hybride (`vector_store`)

La recherche dense interroge ChromaDB avec filtres de métadonnées optionnels (liste de documents, type de chunk). La recherche lexicale s'appuie sur un index BM25 en mémoire reconstruit paresseusement depuis la collection ChromaDB (source de vérité unique) lorsqu'il est marqué obsolète par une ingestion ou une suppression ; ce choix évite un second index persistant à maintenir synchrone, pour un coût de reconstruction négligeable à l'échelle du corpus. La fusion RRF additionne, pour chaque chunk, les $1/(60 + \text{rang})$ de ses positions dans les deux classements, sur un sur-échantillonnage de $3 \times k$ candidats par voie ; les scores fusionnés sont normalisés par le maximum pour l'affichage.

3.3.5 Routage automatique du périmètre (`auto-scope`)

Lorsque l'utilisateur laisse le périmètre en automatique, un appel LLM léger (température 0) reçoit le catalogue *groupes* → *documents indexés* et la question, et répond en JSON strict par la liste des groupes plausiblement pertinents. La conception est volontairement *conservatrice* : toute défaillance (réponse non parsable, groupe inconnu, exception réseau) ou toute réponse

équivalente à « tous les groupes » désactive le filtre : le routage ne peut jamais rétrécir le corpus par accident, seulement l'élaguer quand le modèle est confiant. Les groupes retenus sont affichés à l'utilisateur avec la réponse (« recherché dans : ... »), pour la transparence du périmètre.

3.3.6 Génération contrainte et restitution multimodale

Le prompt système de génération impose : réponse fondée *uniquement* sur les sources numérotées, citations `[Source N]` en ligne, aveu explicite lorsque les sources ne contiennent pas la réponse. Il traite aussi une subtilité multimodale : le modèle est informé que certaines sources sont des figures (dont il ne voit que la description) et que l'application affiche automatiquement l'image citée sous la réponse ; sans cette instruction, le LLM s'excusait de « ne pas pouvoir montrer d'images ». Côté API, chaque source est restituée typée (`text` avec extrait, ou `image` avec URL et description), et le frontend affiche les cartes de sources et les figures citées.

3.3.7 Interface utilisateur

L'interface React comprend trois espaces : l'accueil, l'espace *Documents* (dropzone de téléversement multiple, cartes de documents avec état d'ingestion en direct (étape courante affichée pendant le traitement), vue détail avec rendu Markdown et galerie de figures, gestion des groupes) et l'espace *Chat* (historique des conversations, sélecteur de périmètre documentaire, rendu Markdown des réponses avec `react-markdown`, cartes de sources cliquables, visionneuse de figures). Le thème clair/sombre est persistant et l'ensemble des appels passe par un client API central qui joint le jeton de session Clerk.

3.4 Intégration des services tiers

Table 3.1 Services externes intégrés et leur rôle.

Service	Rôle	Modalités d'intégration
Mistral AI	Embeddings + génération + routage	SDK `mistralai` ; batch d'embeddings normalisés L2 ; température 0 pour le routage JSON
HF Inference Providers (DeepInfra)	Captioning VLM Gemma-3 27B	`huggingface_hub.InferenceClient` ; images en data-URL base64 ; retry sélectif 5xx
Clerk	Authentification	SDK React côté client ; côté backend, vérification RS256 pure via JWKS (aucun appel API Clerk par requête)
Groq	Juge LLM d'évaluation (Llama 3.3 70B)	Utilisé uniquement dans le notebook RAGAS, indépendant du système évalué
Redis / OCI Cache	Cache de lecture des conversations	Client `redis` ou `fakeredis` selon `REDIS_URL` ; fail-open intégral

Un détail d'intégration élégant côté Clerk : la clé publique (`pk_...`) encode en base64 le domaine du Frontend API, qui est aussi l'émetteur des jetons et l'hôte du JWKS ; le backend décode ce domaine au démarrage et n'a besoin d'aucune configuration supplémentaire ni d'appel de découverte.

3.5 Détails techniques spécifiques à l'environnement contraint

3.5.1 Traversée du proxy Zscaler : `truststore`

Le problème des erreurs `CERTIFICATE_VERIFY_FAILED` (section 3.2) a été résolu proprement par le paquet `truststore` : un appel unique `truststore.inject_into_ssl()` à l'import du paquet applicatif, donc avant tout appel réseau, fait utiliser à Python le magasin de certificats de Windows (qui contient les racines ALTEN/Zscaler) au lieu de la liste `certifi` embarquée. Aucune désactivation de vérification TLS, aucune variable d'environnement fragile : la solution est sûre et transparente pour tout le code aval.

3.5.2 Modèles Docling hors ligne

Le CDN de Hugging Face étant bloqué, les modèles de Docling (détection de layout, TableFormer) ont été téléchargés sur une machine non restreinte (`docling-tools models download`), archivés, publiés en artefact de release GitHub (domaine autorisé), puis dépointés sur le poste via `DOCLING_ARTIFACTS_PATH`. Deux guides de mise en place (poste d'entreprise / poste personnel) documentent la procédure pour les repreneurs du projet.

3.5.3 Cache de conversations fail-open

Le cache JSON des lectures de conversations est conçu pour ne *jamais* pouvoir nuire : toute erreur (Redis injoignable, valeur corrompue, exception du client) est interprétée comme un cache absent et l'application continue sur SQLite. Le backend est choisi par `REDIS_URL` : `memory://` (FakeRedis en processus, poste verrouillé), URL Redis réelle (OCI Cache en production), ou vide (désactivé).

3.6 Sécurité applicative

Sans viser la gestion fine des droits (hors périmètre), le système applique les fondamentaux :

- **Authentification forte déléguée** : sessions gérées par Clerk ; le backend vérifie chaque jeton par signature RS256 contre le JWKS de l'émetteur (clés publiques mises en cache), contrôle l'émetteur et l'expiration, et rejette en 401 tout jeton absent ou invalide ;
- **Cloisonnement des données** : toute lecture/écriture de conversation vérifie que la conversation appartient à l'utilisateur du jeton (réponse 404 sinon, sans révéler l'existence de la ressource) ;
- **Validation des entrées** : schémas Pydantic sur toutes les routes, liste blanche d'extensions de fichiers au téléversement, validation des noms de groupes, identifiants générés côté serveur (UUID) ;
- **CORS restrictif** : origines autorisées explicites par configuration ;
- **Secrets hors dépôt** : clés d'API dans `.env` non versionné, avec `.env.example` documenté ;

— **TLS vérifié de bout en bout** : `truststore` maintient la vérification des certificats à travers le proxy d'inspection (plutôt que de la désactiver, réflexe dangereux courant dans ce type d'environnement).

3.7 Tests et validation : évaluation RAGAS

3.7.1 Méthodologie

La qualité du système a été évaluée avec **RAGAS 0.4.3** [8] sur un corpus de trois documents indexés (*BERT*, *Attention Is All You Need*, *Circuit Diagram Symbols*) et six questions de référence rédigées manuellement avec leurs réponses attendues, couvrant les deux familles documentaires (articles de recherche et manuel technique riche en figures).

Deux précautions méthodologiques structurent le protocole :

— **Juge indépendant** : les métriques à base de LLM sont calculées par `llama-3.3-70b-versatile` (servi par Groq), un modèle *étranger* au système évalué (qui génère avec Mistral), afin d'éviter le biais d'auto-évaluation ;

— **Espace vectoriel cohérent** : la métrique *Answer Relevancy*, qui compare des embeddings, utilise `mistral-embed`, les embeddings mêmes du système, pour mesurer la pertinence dans l'espace où le système opère.

Quatre métriques sont retenues : *Faithfulness* (la réponse est-elle fidèle aux passages récupérés, sans hallucination ?), *Answer Relevancy* (répond-elle à la question ?), *Context Precision* (les passages récupérés sont-ils pertinents ?) et *Context Recall* (l'information nécessaire a-t-elle été récupérée ?) ; les deux premières évaluant la génération, les deux dernières le retrieval.

3.7.2 Résultats obtenus

Table 3.2 Résultats de l'évaluation RAGAS (juge : Llama 3.3 70B via Groq).

Question	Faithf.	Ans. Rel.	Ctx. Prec.	Ctx. Rec.
Que signifie l'acronyme BERT ?	1,000	0,890	0,756	1,000
Sur quelles deux tâches BERT est-il pré-entraîné ?	1,000	0,896	0,700	1,000
Quel pourcentage de tokens est masqué lors du MLM ?	1,000	0,959	0,700	1,000
Quel token spécial est ajouté en début de séquence ?	1,000	0,987	1,000	1,000
À quoi sert un schéma de circuit électrique ?	1,000	0,926	0,888	1,000
Que représente le symbole de mise à la terre ?	0,000	0,926	1,000	1,000
Moyenne	**0,833**	**0,931**	**0,841**	**1,000**

3.7.3 Analyse des résultats

— **Context Recall = 1,00 partout** : l'information nécessaire est systématiquement récupérée : la validation la plus directe de l'indexation multimodale unifiée et de la recherche hybride dense + BM25 ;

— **Answer Relevancy moyenne de 0,93** : les réponses adressent bien les questions posées ;
— **Faithfulness parfaite sur 5 questions sur 6** ; le zéro de la question sur le symbole de mise à la terre est instructif : la réponse décrivait l'*apparence* du symbole (fidèle à la description VLM récupérée) plutôt que sa *signification électrique* attendue ; le contexte récupéré, issu d'une description d'image, ne contenait pas la définition sémantique. L'axe d'amélioration identifié porte sur le prompt de captioning et de génération (distinguer « décrire » d'« expliquer la signification ») ;
— **Context Precision de 0,70–0,76 sur les questions BERT** : le retrieval remonte parfois des chunks d'*Attention Is All You Need* non pertinents pour BERT, les deux articles partagent un vocabulaire très proche. Pistes : re-ranking ou filtrage par document.

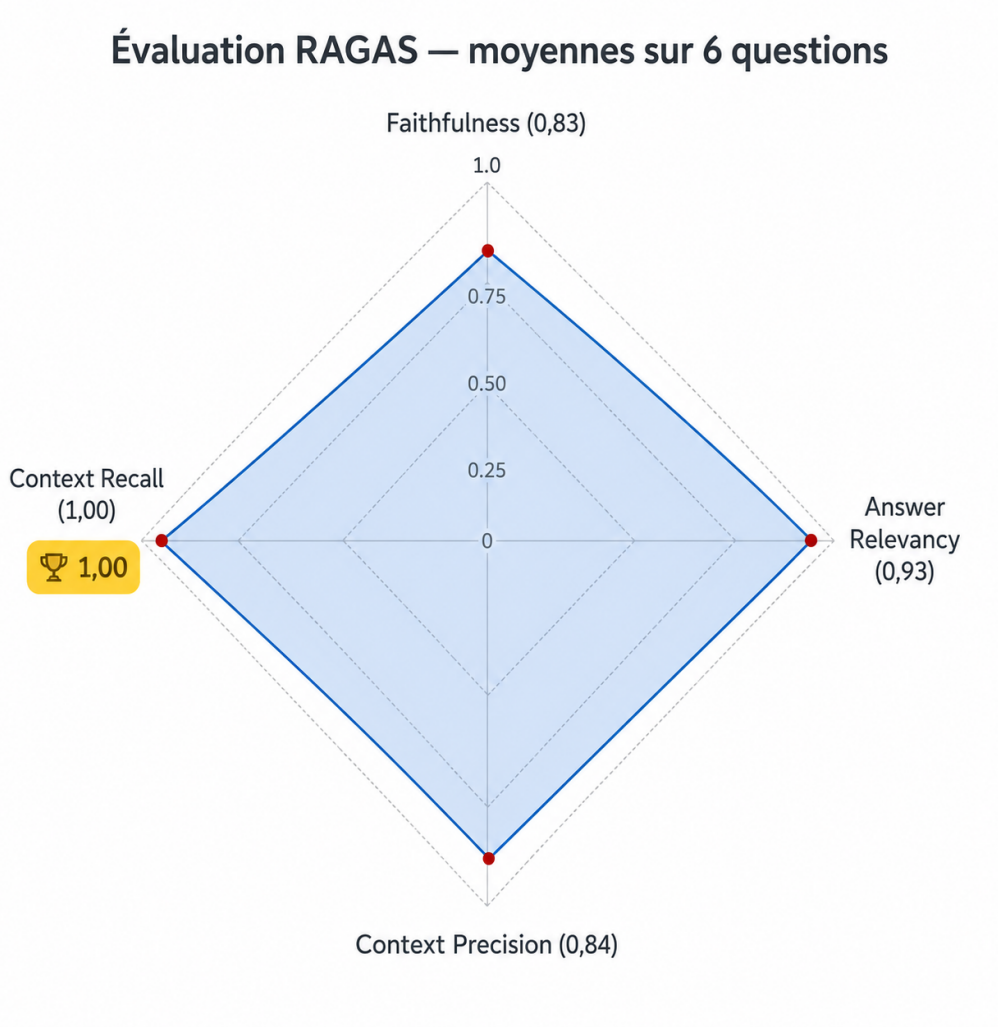


Figure 3.1 Synthèse visuelle des quatre métriques RAGAS moyennes du système.

3.8 Scénario de démonstration

Le scénario de démonstration de fin de stage déroule le cycle complet :

1. Connexion (Clerk), arrivée sur l'espace documentaire vide ;

2. Téléversement en rafale de plusieurs PDF (article BERT + manuel de schémas électriques) ; observation du traitement séquentiel et des étapes d'ingestion en direct (« Describing figure 3 of 5... ») ;
3. Ouverture du détail d'un document : Markdown structuré (tableaux reconstruits), galerie des figures avec descriptions générées par le VLM ;
4. Question textuelle (« Sur quelles tâches BERT est-il pré-entraîné ? ») : réponse citée `[Source 1]` `[Source 2]` avec extraits et pages ;
5. Question visuelle (« Montre-moi l'architecture du Transformer ») : la réponse cite une source de type figure et l'image s'affiche sous la réponse ;
6. Question avec périmètre automatique : l'interface indique les groupes retenus par le routeur LLM ;
7. Question hors corpus : le système déclare explicitement ne pas trouver la réponse dans les sources : démonstration du garde-fou anti-hallucination.

3.9 Résultats obtenus

Au terme des huit semaines, l'ensemble des objectifs du cahier des charges est atteint :

— une chaîne d'ingestion multimodale complète et robuste (PDF/Word/Excel, figures décrites et indexées, file sérialisée, suivi d'état, gestion d'erreurs) ;
— un moteur de restitution hybride (dense + BM25 + RRF) avec routage automatique du périmètre et génération contrainte citée ;
— une API REST documentée (OpenAPI) d'une quinzaine de routes couvrant documents, groupes, conversations et chat ;
— une interface web complète et soignée (authentification, gestion du corpus, chat multi-conversations, affichage des sources et figures, thème clair/sombre) ;
— une évaluation quantitative RAGAS aux résultats solides (recall 1,00 ; relevancy 0,93 ; precision 0,84 ; faithfulness 0,83) avec analyse des cas d'échec ;
— une base prête pour le déploiement OCI (stockage embarqué, cache commutable, configuration par environnement).

3.10 Difficultés rencontrées

Blocages réseau successifs. La difficulté dominante du stage. L'inspection TLS de Zscaler a d'abord cassé toutes les requêtes HTTPS Python (résolu par `truststore`) ; puis le blocage du CDN Hugging Face a empêché le téléchargement des modèles Docling (résolu par le circuit hors ligne via release GitHub) ; enfin le blocage de l'API d'inférence HF a invalidé le backend d'embeddings initial, imposant la bascule vers `mistral-embed`, et donc la purge et la réingestion complète de l'index (espaces vectoriels incompatibles à dimension égale) ;

Robustesse du parsing PDF. Le backend Docling par défaut s'effondrait en `std::bad_alloc`

sur des PDF à images volumineuses ; le passage au backend `pypdfium2` a réglé le problème au prix d'une fidélité de layout légèrement moindre ;

Contention d'écriture sous téléversements en rafale. Les ingestions concurrentes (une par requête via `BackgroundTasks`) saturaient le poste et faisaient échouer des documents par contention SQLite/ChromaDB, résolu par la file à worker unique ;

Comportement du LLM face aux sources visuelles. Sans instruction dédiée, le modèle s'excusait de ne pas pouvoir afficher d'images (alors que l'application le fait pour lui) ; et les descriptions VLM initiales, trop « visuelles », ont motivé le prompt d'ancrage contextuel et le préfixage des légendes d'origine ;

Calendrier contraint. Huit semaines pour l'ensemble de la chaîne : les choix de sobriété (pas de LangChain, stockage embarqué, services managés) ont été autant des choix de qualité que des conditions de faisabilité.

3.11 Limites de la solution

— **Passage à l'échelle du retrieval** : l'index BM25 est reconstruit en mémoire depuis ChromaDB, approprié à un corpus de démonstration, à revoir (index persistant, base dédiée) pour des milliers de documents ;
— **Pas d'OCR** : les documents scannés (images de texte) ne sont pas couverts par cette itération ; l'architecture le permet (option Docling), la contrainte d'installation de Tesseract ou le coût d'un service cloud l'ont différé ;
— **Dépendance aux API externes** : embeddings, captioning et génération reposent sur des services tiers : disponibilité, latence et limites de débit s'imposent au système ;
— **Évaluation à petite échelle** : six questions de référence suffisent à diagnostiquer, pas à qualifier statistiquement ; un jeu d'évaluation étendu et automatisé serait la suite naturelle ;
— **Corpus partagé entre utilisateurs** : conformément au périmètre, seules les conversations sont cloisonnées par utilisateur ; le corpus documentaire est commun (pas de gestion fine des droits).

3.12 Conclusion

Ce chapitre a montré la traduction de l'architecture en un système opérationnel, validé quantitativement, et façonné en profondeur par son environnement d'exécution : chaque contrainte du poste verrouillé a trouvé une réponse d'ingénierie propre (truststore, modèles hors ligne, backends commutables, stockage embarqué, file sérialisée) plutôt qu'un contournement fragile. Les résultats RAGAS valident les deux paris techniques centraux, l'indexation multimodale par description VLM et la recherche hybride RRF, tout en identifiant avec précision les axes d'amélioration.

Conclusion générale

Ce stage de Projet de Fin d'Année avait pour ambition de rendre interrogeable en langage naturel un corpus de documentation technique, *y compris son contenu visuel*, et de restituer des réponses fiables et vérifiables. Cet objectif est atteint : l'assistant RAG multimodal développé couvre l'intégralité de la chaîne, du téléversement d'un PDF brut à la réponse citée illustrée par la figure pertinente.

Sur le plan technique, le projet démontre la maîtrise d'un spectre large et actuel : analyse documentaire par apprentissage profond (Docling, TableFormer), découpage sémantique contextualisé, description de figures par modèle de vision-langage (Gemma-3 27B), représentations vectorielles et bases vectorielles (mistral-embed, ChromaDB), recherche hybride dense/lexicale fusionnée par RRF, ingénierie de prompts pour la génération contrainte et le routage (Mistral), développement d'API REST (FastAPI, Pydantic), authentification moderne (Clerk, JWT/JWKS), interface web réactive (React, TypeScript, Vite) et évaluation quantitative de systèmes RAG (RAGAS, juge LLM indépendant). Les résultats de l'évaluation (rappel de contexte parfait, pertinence des réponses de 0,93) valident les choix d'architecture.

Sur le plan de l'ingénierie, la leçon la plus formatrice du stage tient peut-être à son environnement : développer un système dépendant de modèles d'IA distants sur un poste verrouillé, derrière un proxy d'inspection TLS, a imposé de diagnostiquer des pannes réseau opaques, d'arbitrer des changements de fournisseurs en cours de projet (bascule d'embeddings avec réindexation complète), et de concevoir chaque composant pour la résilience et la commutabilité. Ces compétences, comprendre *pourquoi* une infrastructure échoue et concevoir *avec* les contraintes plutôt que contre elles, sont précisément celles qu'exige l'industrialisation de l'IA en entreprise.

Enfin, ce travail livre à ALTEN un démonstrateur complet et documenté, dont l'architecture sobre (stockage embarqué, services managés, configuration par environnement) est directement transposable sur Oracle Cloud Infrastructure.

Perspectives

Plusieurs prolongements, identifiés pendant le stage et volontairement hors périmètre, constituent des suites naturelles :

— **Déploiement OCI complet** : conteneurisation du backend et du frontend sur une VM OCI, cache basculé sur OCI Cache, stockage des fichiers sur Object Storage, et éventuellement migration des métadonnées vers Oracle Autonomous Database (palier Always Free) ;

— **Re-ranking** : l'ajout d'un ré-ordonnanceur (cross-encoder ou LLM) sur les candidats du retrieval hybride est la réponse directe aux scores de *context precision* observés sur les corpus à vocabulaire proche ;

— **OCR des documents scannés** : activation de l'option OCR de Docling ou recours à un service managé (Azure Document Intelligence), pour couvrir les documents-images ;

— **Captioning orienté signification** : itération sur le prompt du VLM pour distinguer la description visuelle d'une figure de l'explication de sa signification technique, l'axe identifié par le cas d'échec de faithfulness ;

— **Jeu d'évaluation étendu et continu** : constitution d'un jeu de questions de référence plus large (génération assistée puis validation humaine) et intégration de l'évaluation RAGAS en contrôle continu de non-régression ;

— **Streaming des réponses** : restitution token par token (SSE) pour améliorer la latence perçue ;

— **Embeddings multimodaux natifs** : à plus long terme, une voie ColPali (late interaction sur pages entières) pourrait compléter ou remplacer le captioning pour les corpus où la mise en page elle-même est porteuse de sens ;

— **Gestion des droits documentaires** : cloisonnement du corpus par utilisateur ou par équipe, prérequis d'un déploiement au-delà du démonstrateur.

Bibliographie

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, *Attention Is All You Need*, Advances in Neural Information Processing Systems (NeurIPS), 2017.

[2] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, *BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding*, NAACL-HLT, 2019.

[3] P. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS, 2020.

[4] S. Robertson, H. Zaragoza, *The Probabilistic Relevance Framework : BM25 and Beyond*, Foundations and Trends in Information Retrieval, vol. 3, n°4, 2009.

[5] G. V. Cormack, C. L. A. Clarke, S. Büttcher, *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*, SIGIR, 2009.

[6] A. Radford et al., *Learning Transferable Visual Models From Natural Language Supervision* (CLIP), ICML, 2021.

[7] M. Faysse et al., *ColPali : Efficient Document Retrieval with Vision Language Models*, arXiv :2407.01449, 2024.

[8] S. Es, J. James, L. Espinosa-Anke, S. Schockaert, *RAGAS : Automated Evaluation of Retrieval Augmented Generation*, arXiv :2309.15217, 2023.

[9] IBM Research, *Docling : Document conversion and understanding toolkit*, https://docling-project.github.io/docling/, consulté en juin 2026.

[10] Chroma, *ChromaDB : the AI-native open-source embedding database*, https://docs.trychroma.com/, consulté en juin 2026.

[11] Mistral AI, *Mistral AI Platform Documentation : Embeddings & Chat Completions*, https://docs.mistral.ai/, consulté en juin 2026.

[12] Google DeepMind, *Gemma 3 Technical Report*, 2025. https://ai.google.dev/gemma, consulté en juin 2026.

[13] S. Ramírez, *FastAPI Documentation*, https://fastapi.tiangolo.com/, consulté en juin 2026.

[14] Meta Open Source, *React Documentation*, https://react.dev/, consulté en juin 2026.

[15] Clerk Inc., *Clerk Documentation : Authentication and User Management*, https://clerk.com/docs, consulté en juin 2026.

[16] S. Seth (PSF), *truststore : Verify certificates using OS trust stores*, https://truststore.readthedocs.io/, consulté en juin 2026.

[17] Exploding Gradients, *RAGAS Documentation*, https://docs.ragas.io/, consulté en juillet 2026.

A Chapitre

Principales routes de l'API REST

Table A.1 Routes exposées par l'API FastAPI.

Méthode	Route	Rôle
POST	`/api/documents/upload`	Téléverser un document (PDF/DOCX/XLSX) et déclencher son ingestion
GET	`/api/documents`	Lister les documents et leur état d'ingestion
GET	`/api/documents/{id}`	Détail d'un document : métadonnées, Markdown, figures décrites
GET	`/api/documents/{id}/download`	Télécharger le fichier source
PATCH	`/api/documents/{id}/name`	Renommer un document
PATCH	`/api/documents/{id}/category`	Changer le groupe d'un document
POST	`/api/documents/{id}/duplicate`	Dupliquer un document (contenu partagé, sans recopie)
DELETE	`/api/documents/{id}`	Supprimer un document (fichier, Markdown, images, vecteurs)
GET / POST	`/api/groups`	Lister / créer les groupes documentaires
PATCH / DELETE	`/api/groups/{id}`	Renommer / supprimer un groupe (repli des documents)
POST	`/api/chat`	Poser une question (périmètre manuel ou automatique) et recevoir la réponse sourcée
GET	`/api/conversations`	Lister les conversations de l'utilisateur authentifié
GET	`/api/conversations/{id}`	Recharger une conversation et ses messages
GET	`/static/images/...`	Servir les figures extraites (affichage des sources)